Airflow

What is the Local Executor?

- Run multiple tasks in parallel
- Based on the Python module: multiprocessing
- Spawn processes to execute tasks
- Easy to set up
- Resource light
- Vertical Scaling
- Single point of failure
- As a best practice, always try the Local Executor first! KISS (Keep it simple, stupid)

Parallel Execution enables simultaneous execution of tasks, which enhances performance and reduces the time required for task completion.

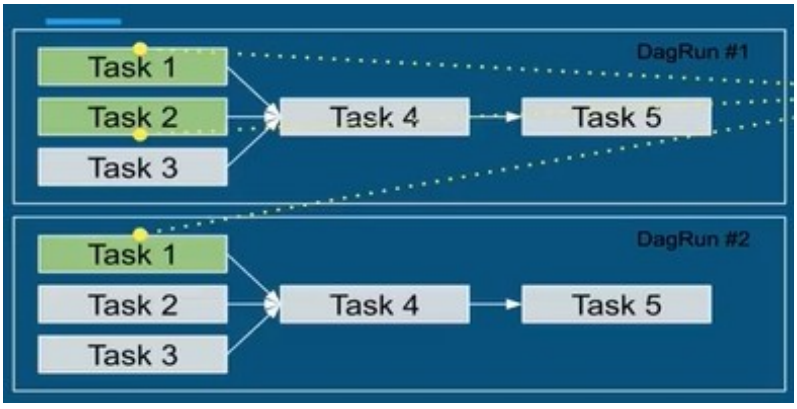
What is PostgreSQL?

PostgreSQL is an object-relational database management system that executes using SQL, being used in t

- Object-relational database
- ACID compliant
- Client-server database
- Handle concurrent connections
- SQL standard and advanced
- Highly scalable
- Extensible

There's a parameter for concurrency and parallelism: If parallelism=3, 3 tasks can be run simultaneously FOR ALL DAGRuns.

If max_active_runs_per_dag = 1, then only DAGRun #1 will run, once finished, then DAGRun #2 is run, just as following in the image below:



If dag_concurrency=2 up to 2 tasks can be run PER DAGRun. In the DagRun #1, the third task can't be run simultaneously since dag_concurrency=2.

Local Executor strategies

- parallelism = 0
  - "Unlimited" parallelism
- parallelism > 0
  - Limited parallelism

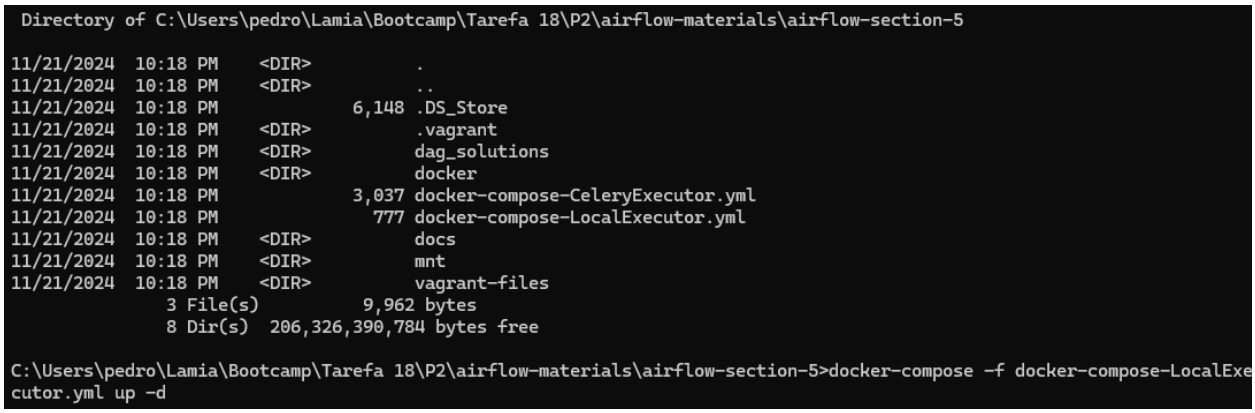
airflow-materials > airflow-section-5 > docker-compose-LocalExecutor.yml

```yaml
version: '2.1'
services:
    postgres:
        image: postgres:9.6
        environment:
            - POSTGRES_USER=airflow
            - POSTGRES_PASSWORD=airflow
            - POSTGRES_DB=airflow
    webserver:
        build: ./docker/airflow
        restart: always
        depends_on:
            - postgres
        environment:
            - LOAD_EX=n
            - EXECUTOR=Local
        volumes:
            - ./mnt/airflow/dags:/usr/local/airflow/dags
            - ./mnt/airflow/airflow.cfg:/usr/local/airflow/airflow.cfg
        ports:
            - "8080:8080"
        command: webserver
        healthcheck:
            test: ["CMD-SHELL", "[ -f /usr/local/airflow/airflow-webserver.pid ]"]
            interval: 30s
            timeout: 30s
            retries: 3

```

This makes so only one task can be run in parallel.

airflow-materials > airflow-section-5 > mnt > airflow > airflow.cfg

```
# The amount of parallelism as a setting to the executor. This defines
# the max number of task instances that should run simultaneously
# on this airflow installation
parallelism = 1
```



Airflow Metadata Database and Local Executor Configuration

I explored how to interact with a Postgres database, query metadata, and manage the Airflow environment. Additionally, security considerations were emphasized, particularly regarding the configuration of the sql_mode parameter in the airflow.cfg file. Best practices for securing the database while using Airflow were discussed, with a recommendation to keep Airflow focused on orchestration rather than database exploration.

Starting the Web Server:

To start the web server, the following command is used:

```
docker-compose -f docker-compose-LocalExecutor.yml up -d
```

In this case, the docker-compose function is basing its process in the docker-compose-LocalExecutor.yml file.

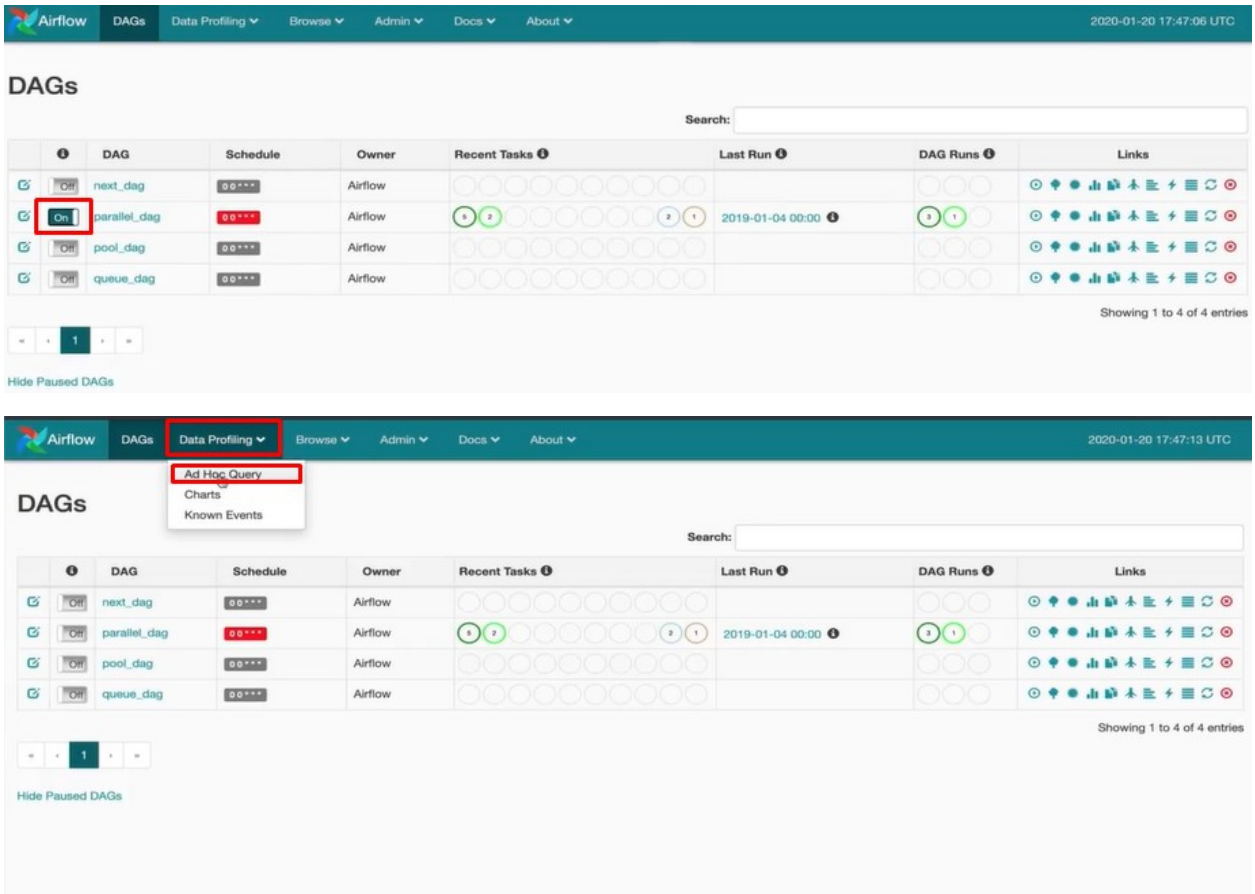
Docker and Postgres Configuration Setup

I began by setting up the local executor with Postgres using a Docker Compose configuration. The docker-compose-LocalExecutor.yml file defines two services:

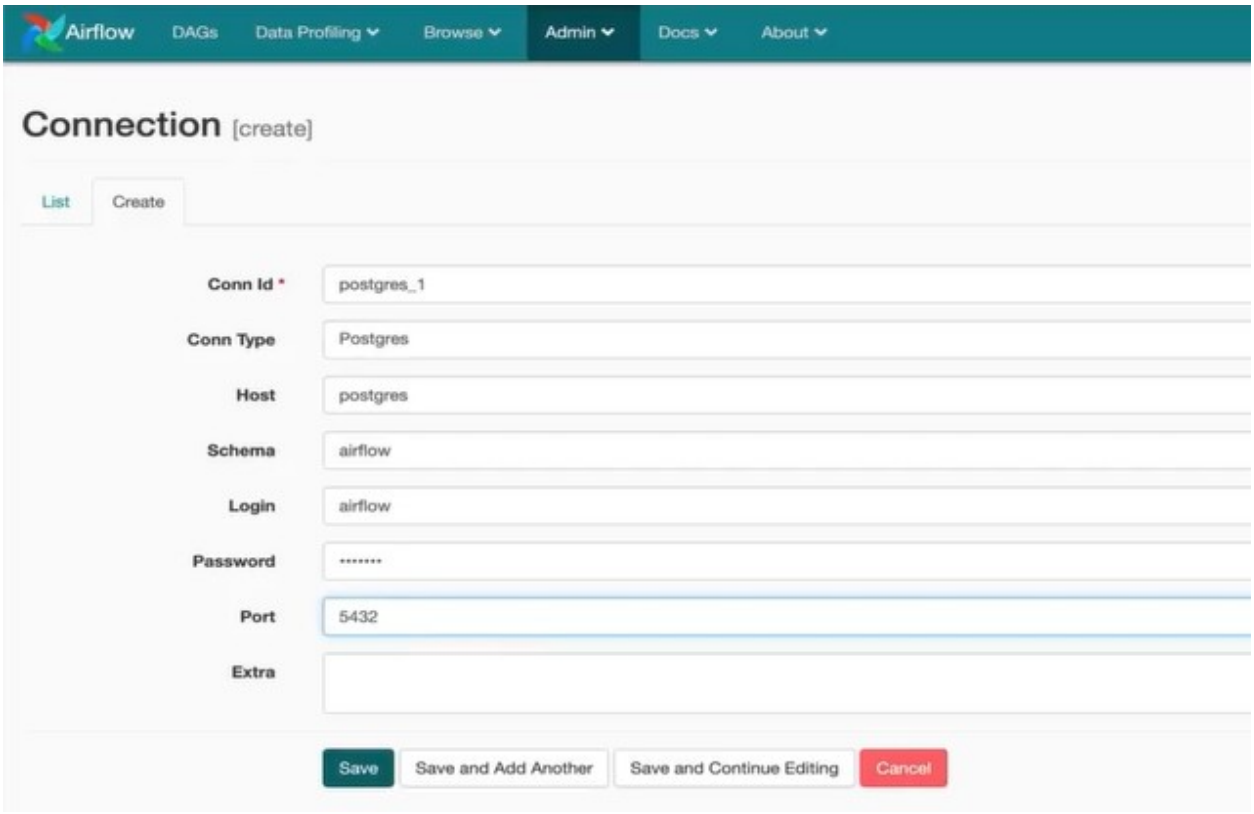
- Postgres: The database is configured using the Postgres 9.6 image, with environment variables for the user, password, and database name.
- Webserver: This service depends on Postgres and includes the Airflow configuration, such as the local executor setup, port exposure, and health checks. It's important to note that both the scheduler and web server run in the same container in this configuration.

Getting data

Now, airflow can be opened in localhost:8080. A task is set to run just so it can generate some data to explore:

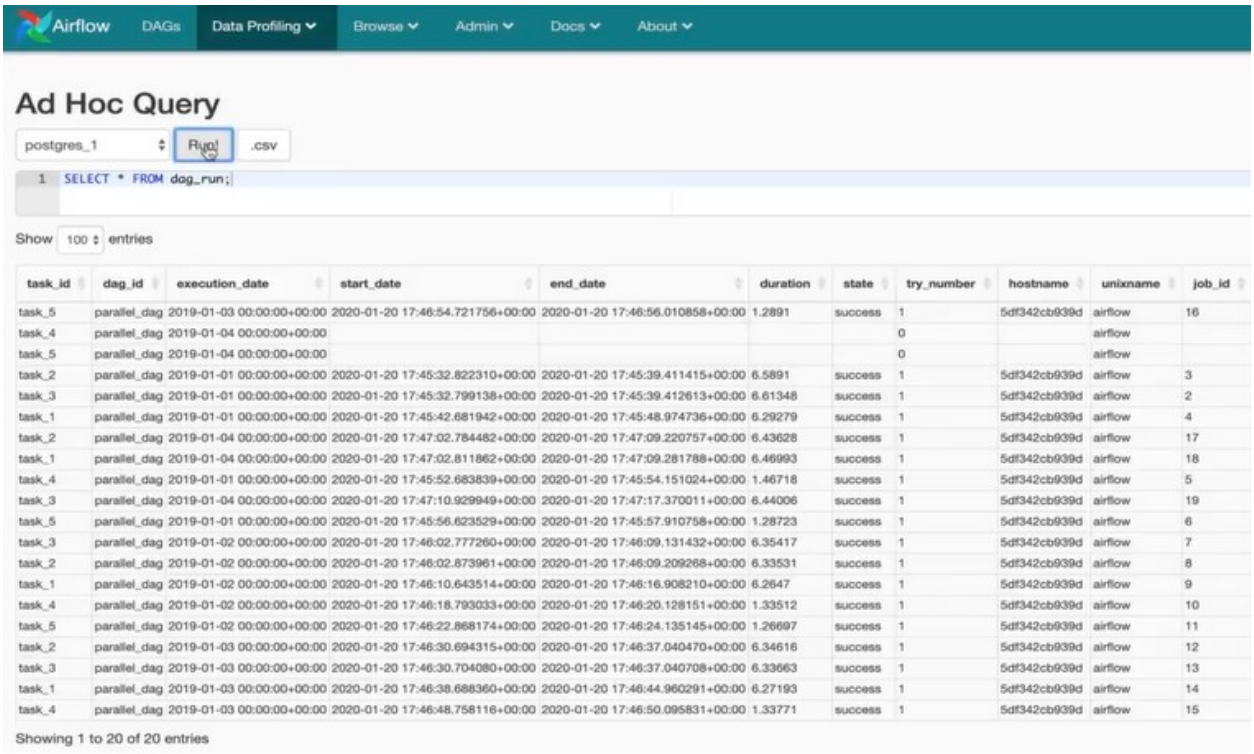


But first, a connection must be created:



Note: all the configuration put in that connection was based from the entrypoint.sh file.

After it, SQL commands can be used into Ad Hoc Query:



Now it's just a matter of criativity to enter the needed commands.

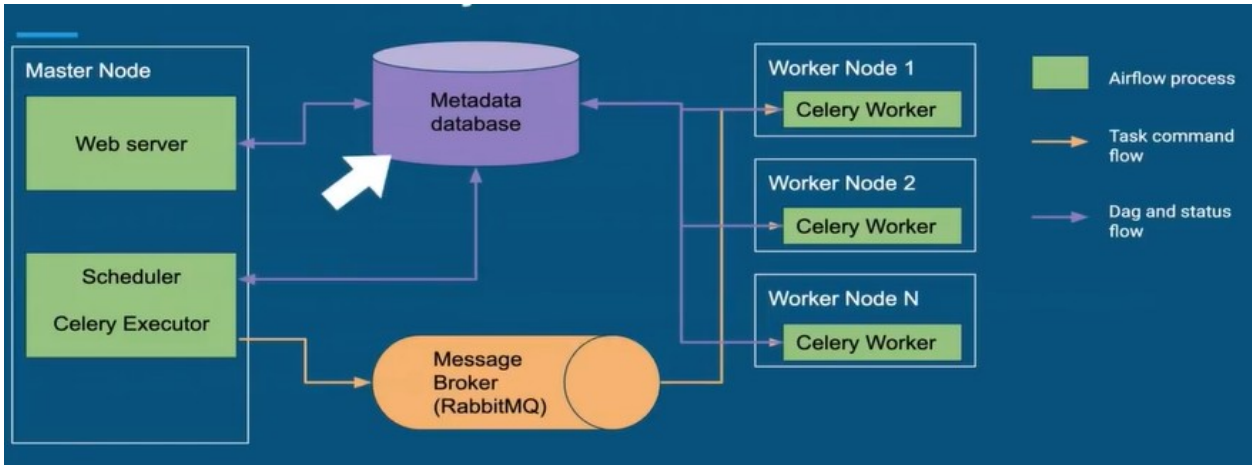
Scaling out Apache Airflow with Celery Executors

What is the Celery Executor?

- Scale out Apache Airflow
- Backed by Celery: Asynchronous Distributed Task Queue
- Distribute tasks among worker nodes
- `airflow worker`: Create a worker to execute tasks
- Horizontal Scaling
- High availability: If one worker goes down, Airflow can still schedule tasks
- Need a message broker: RabbitMQ/Redis
- Can set different queues

The Celery Executor is an excellent choice for scaling Apache Airflow in distributed environments. It enables horizontal scaling by distributing tasks across multiple worker nodes, ensuring efficient resource utilization. With high availability, Airflow can continue scheduling tasks even if individual workers fail, making it highly reliable. Additionally, it supports message brokers like RabbitMQ or Redis for communication and allows the configuration of different task queues, offering flexibility in workload management. This setup is ideal for handling complex workflows with high throughput and varying resource demands.

Here is a chart on how it works:



What are the drawbacks?

- Need an extra component: Message Broker Service
- Take time and work to set up
- Worker maintenance

In the file "docker-compose-CeleryExecutor.yml", it is possible to look at the different services:

airflow-materials > airflow-section-5 > docker-compose-CeleryExecutor.yml

```yaml
version: '2.1'
services:
    redis:
        image: 'redis:5.0.5'
        command: redis-server --requirepass redispass

    postgres:
        image: postgres:9.6
        environment:
            - POSTGRES_USER=airflow
            - POSTGRES_PASSWORD=airflow
            - POSTGRES_DB=airflow
        # Uncomment these lines to persist data on the local filesystem.
        #     - PGDATA=/var/lib/postgresql/data/pgdata
        # volumes:
        #     - ./mnt/postgres:/var/lib/postgresql/data/pgdata

    webserver:
        build: ./docker/airflow
        restart: always
        depends_on:
            - postgres
            - redis
        environment:
            - LOAD_EX=n
            - FERNET_KEY=46BKJoQYlPPOexq0OhDZnIlNepKFf87WFwLbfzqDDho=
            - EXECUTOR=Celery
            - POSTGRES_USER=airflow
            - POSTGRES_PASSWORD=airflow
            - POSTGRES_DB=airflow
            - REDIS_PASSWORD=redispass
        volumes:
            - ./mnt/airflow/dags:/usr/local/airflow/dags
            - ./mnt/airflow/airflow.cfg:/usr/local/airflow/airflow.cfg
            # Uncomment to include custom plugins
            # - ./plugins:/usr/local/airflow/plugins
        ports:
            - "8080:8080"
        command: webserver
        healthcheck:
            test: ["CMD-SHELL", "[ -f /usr/local/airflow/airflow-webserver.pid ]"]
            interval: 30s
            timeout: 30s
            retries: 3

    flower:
        build: ./docker/airflow
        restart: always
```

In the image, it's possible to see we're using redis as the message broker, for example.

In the image below, it shows the web server using a `FERNET_KEY`, which is used to encrypt the connections, will be better explained later on.

```
webserver:
    build: ./docker/airflow
    restart: always
    depends_on:
        - postgres
        - redis
    environment:
        - LOAD_EX=n
        - FERNET_KEY=46BKJoQYlPPOexq0OhDZnIlNepKFf87WFwLbfzqDDho=
        - EXECUTOR=Celery
        - POSTGRES_USER=airflow
        - POSTGRES_PASSWORD=airflow
        - POSTGRES_DB=airflow
        - REDIS_PASSWORD=redispass
```

To compose using the file above:

`docker-compose -f docker-compose-CeleryExecutor.yml up -d`

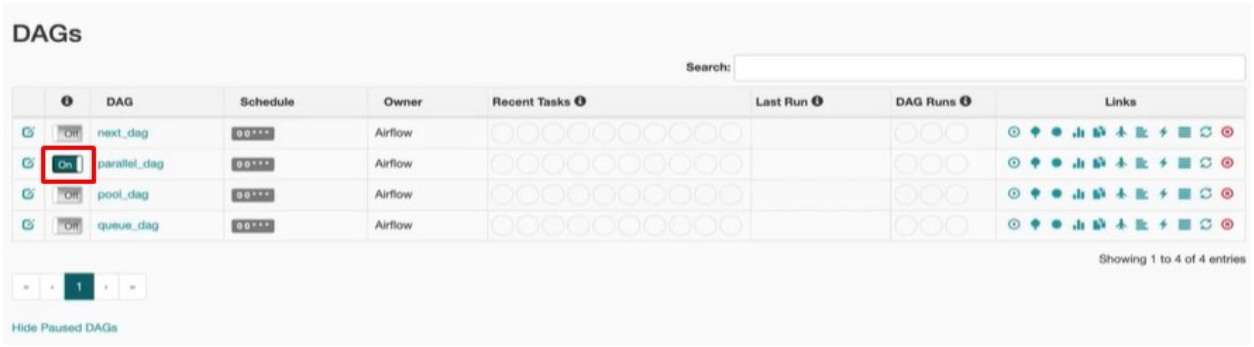
```
Marc@MacBook-Pro ~/airflow-materials/airflow-section-5 (master) $ docker-compos
e -f docker-compose-CeleryExecutor.yml up -d
Creating network "airflow-section-5_default" with the default driver
Pulling redis (redis:5.0.5)...
5.0.5: Pulling from library/redis
b8f262c62ec6: Pull complete
93789b5343a5: Pull complete
49cdbb315637: Pull complete
2c1ff453e5c9: Pull complete
9341ee0a5d4a: Pull complete
770829e1df34: Pull complete
Digest: sha256:5dcccb533dc0deacce4a02fe9035134576368452db0b4323b98a4b2ba2d3b302
Status: Downloaded newer image for redis:5.0.5
Creating airflow-section-5_redis_1    ... done
Creating airflow-section-5_postgres_1 ... done
Creating airflow-section-5_flower_1    ... done
Creating airflow-section-5_webserver_1 ... done
Creating airflow-section-5_scheduler_1 ... done
Creating airflow-section-5_worker_1    ... done
```

Besides the airlfow server on the port 8080, it also opens the port for the flower server, on the port 5555:

```yaml
webserver:
    build: ./docker/airflow
    restart: always
    depends_on:
        - postgres
        - redis
    environment:
        - LOAD_EX=n
        - FERNET_KEY=46BKJoQYlPPOexq0OhDZnI
        - EXECUTOR=Celery
        - POSTGRES_USER=airflow
        - POSTGRES_PASSWORD=airflow
        - POSTGRES_DB=airflow
        - REDIS_PASSWORD=redispass
    volumes:
        - ./mnt/airflow/dags:/usr/local/air
        - ./mnt/airflow/airflow.cfg:/usr/lo
        # Uncomment to include custom plugi
        # - ./plugins:/usr/local/airflow/pl
    ports:
        - "8080:8080"
    command: webserver
    healthcheck:
        test: ["CMD-SHELL", "[ -f /usr/loca
        interval: 30s
        timeout: 30s
        retries: 3

flower:
    build: ./docker/airflow
    restart: always
    depends_on:
        - redis
    environment:
```

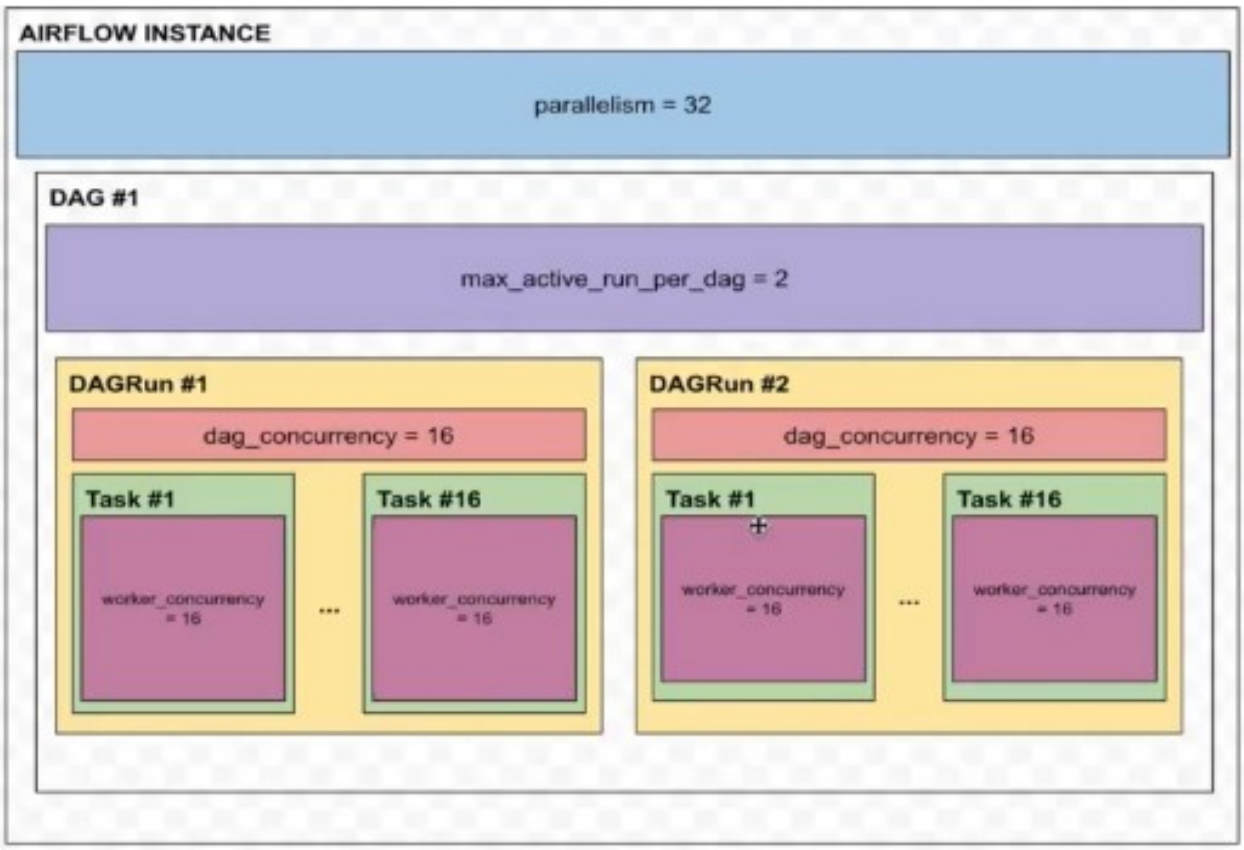
Here is the worker:

Now entering airflow and starting a DAG:



In the flower dashboard, it can be seen that the worker keeps track of the state of each task that is being executed in airflow, providing some interesting labels such as "Active", "Processed", "Failed", "Succeeded", "Retried" and "Load Average":

"Load Average" keeps track of how intensely the CPU is being used to process all of the tasks, being useful for some insights in



How to increase the number of executed tasks to the worker node

To do it, it's needed to add new nodes to the cluster:

It's possible to do it manually, making it easy to set it regardless on which architecture is being worked on.

When using the docker-compose method, all the containers started are able to comunicate with each other.

To do it manually, it's needed to expose the ports of the desired container:

```
Marc@MacBook-Pro ~/airflow-materials/airflow-section-5 (master) $ docker-compos
e -f docker-compose-CeleryExecutor.yml up -d
Creating network "airflow-section-5_default" with the default driver
Creating airflow-section-5_postgres_1 ... done
Creating airflow-section-5_redis_1    ... done
Creating airflow-section-5_flower_1   ... done
Creating airflow-section-5_webserver_1 ... done
Creating airflow-section-5_scheduler_1 ... done
Creating airflow-section-5_worker_1   ... done
```

```
docker run --network airflow-section-5_default --expose 8793 -v
./mnt/airflow/dags:/usr/local/airflow/dags -v
./mnt/airflow/airflow.cfg:/usr/local/airflow/airflow.cfg python:3.7
```

Since the `-v` argument does not support relative paths, so wherever there's a "." to generalize a path, it should be substituted by the actual path in the computer.

After it, in the airflow.cfg file, the parameters "executor" and "sql_alchemy_conn" should be changed for "CeleryExecutor" and "postgresql+psycopg2://airflow:airflow@postgres:5432/airflow", respectively.

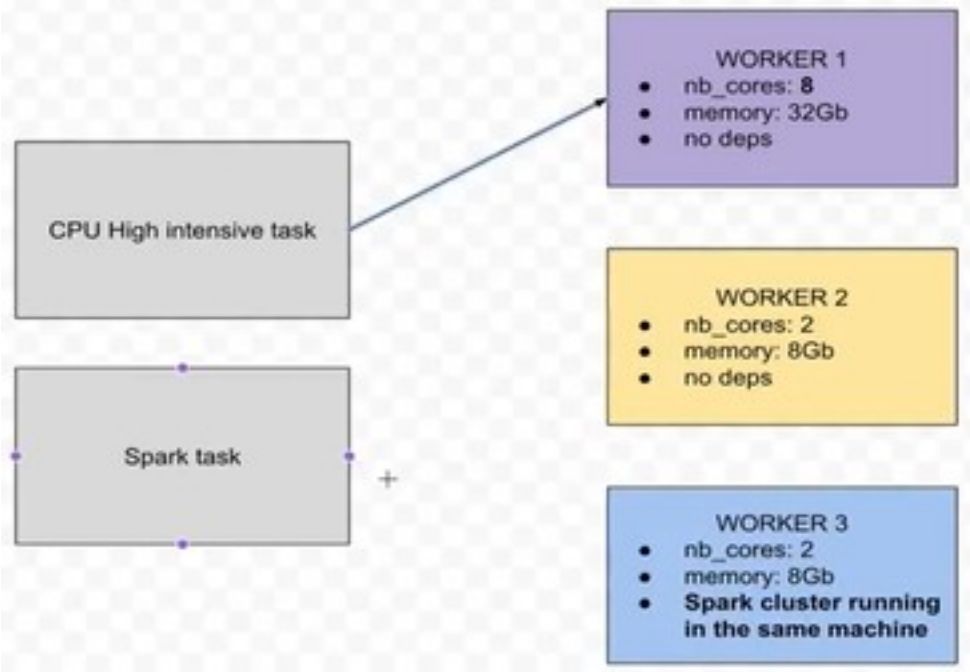
```
# The executor class that airflow should use. Choices include
# SequentialExecutor, LocalExecutor, CeleryExecutor, DaskExecutor, KubernetesExecu
executor = SequentialExecutor

# The SqlAlchemy connection string to the metadata database.
# SqlAlchemy supports many different database engine, more information
# their website
sql_alchemy_conn = sqlite:////usr/local/airflow/airflow.db
```

```
# The executor class that airflow should use. Choices include
# SequentialExecutor, LocalExecutor, CeleryExecutor, DaskExecutor, KubernetesExecu
executor = CeleryExecutor

# The SqlAlchemy connection string to the metadata database.
# SqlAlchemy supports many different database engine, more information
# their website
sql_alchemy_conn = postgresql+psycopg2://airflow:airflow@postgres:5432/airflow
```

After it, the new worker node will be already working:

Sending tasks to specific workers

It may be useful in the case of letting a specific worker with more dedicated ram/cpu to deal with a more intensive task, to optimize the system.




```
# Default queue that tasks get assigned to and that worker listen on.
default_queue = default
```

Creating 3 different queues, each one with an specific metric.

```
worker:
    build: ./docker/airflow
    restart: always
    depends_on:
        - scheduler
    volumes:
        - ./mnt/airflow/dags:/usr/local/airflow/dags
        - ./mnt/airflow/airflow.cfg:/usr/local/airflow/airflow.cfg
        # Uncomment to include custom plugins
        # - ./plugins:/usr/local/airflow/plugins
    environment:
        - FERNET_KEY=46BKJoQYlPPOexq0OhDZnIlNepKFf87WFwLbfzqDDho=
        - EXECUTOR=Celery
        - POSTGRES_USER=airflow
        - POSTGRES_PASSWORD=airflow
        - POSTGRES_DB=airflow
        - REDIS_PASSWORD=redispass
    command: worker
```

```
worker:
    build: ./docker/airflow
    restart: always
    depends_on:
        - scheduler
    volumes:
        - ./mnt/airflow/dags:/usr/local/airflow/dags
        - ./mnt/airflow/airflow.cfg:/usr/local/airflow/airflow.cfg
        # Uncomment to include custom plugins
        # - ./plugins:/usr/local/airflow/plugins
    environment:
        - FERNET_KEY=46BKJoQYlPPOexq0OhDZnIlNepKFf87WFwLbfzqDDho=
        - EXECUTOR=Celery
        - POSTGRES_USER=airflow
        - POSTGRES_PASSWORD=airflow
        - POSTGRES_DB=airflow
        - REDIS_PASSWORD=redispass
    command: worker -q worker_ssd, worker_cpu, worker_spark
```

Now in the terminal:

`docker-compose -f docker-compose-CeleryExecutor.yml up -d --scale worker=3`

```
Marc@MacBook-Pro ~/airflow-materials/airflow-section-5 (master) $ docker-compos
e -f docker-compose-CeleryExecutor.yml up -d --scale worker=3
Creating network "airflow-section-5_default" with the default driver
Creating airflow-section-5_postgres_1 ... done
Creating airflow-section-5_redis_1    ... done
Creating airflow-section-5_webserver_1 ... done
Creating airflow-section-5_flower_1    ... done
Creating airflow-section-5_scheduler_1 ... done
Creating airflow-section-5_worker_1    ... done
Creating airflow-section-5_worker_2    ... done
Creating airflow-section-5_worker_3    ... done
Marc@MacBook-Pro ~/airflow-materials/airflow-section-5 (master) $
```

Worker Name	Status	Active	Processed	Failed	Succeeded	Retried	Load Average
celery@6dafb601ed9e	Online	0	0	0	0	0	0.62, 0.31, 0.25
celery@29ea6008c6da	Online	0	0	0	0	0	0.62, 0.31, 0.25
celery@85b9336a3fb6	Online	0	0	0	0	0	0.62, 0.31, 0.25

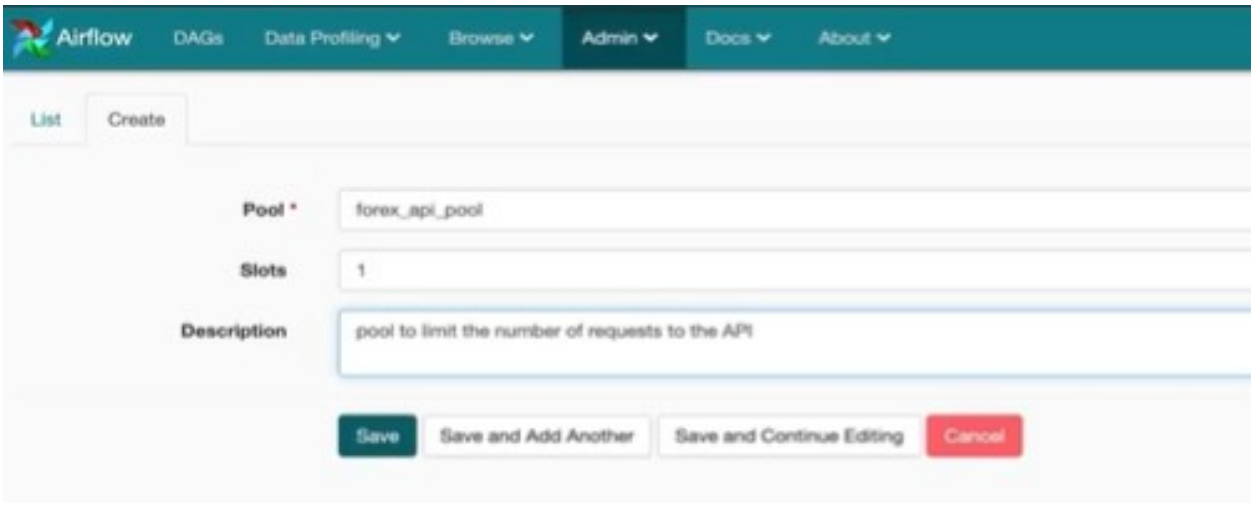
In the DAG configuration file, it's now possible to separate each task for each specific node:

```python
with DAG(dag_id='queue_dag', schedule_interval='0 0 * * *', default_args=default_args, catchup=False) as dag:

    t_1_ssd = BashOperator(task_id='t_1_ssd', bash_command='echo "I/O intensive task"', queue='worker_ssd')

    t_2_ssd = BashOperator(task_id='t_2_ssd', bash_command='echo "I/O intensive task"', queue='worker_ssd')

    t_3_ssd = BashOperator(task_id='t_3_ssd', bash_command='echo "I/O intensive task"', queue='worker_ssd')

    t_4_cpu = BashOperator(task_id='t_4_cpu', bash_command='echo "CPU instensive task"', queue='worker_cpu')

    t_5_cpu = BashOperator(task_id='t_5_cpu', bash_command='echo "CPU instensive task"', queue='worker_cpu')

    t_6_spark = BashOperator(task_id='t_6_spark', bash_command='echo "Spark dependency task"', queue='worker_spark')

    task_7 = DummyOperator(task_id='task_7')

    [t_1_ssd, t_2_ssd, t_3_ssd, t_4_cpu, t_5_cpu, t_6_spark] >> task_7
```

Pools

In Airflow, pools and priority_weights are mechanisms to manage task parallelism and prioritize execution.

Pools limit the number of tasks that can run concurrently within a specific resource group, preventing overload on external systems or shared resources. Each task is assigned to a pool, and the number of available "slots" determines how many tasks from that pool can run at the same time. priority_weights define the execution priority of tasks within the same pool or DAG. Tasks with higher weights are picked first by the scheduler when multiple tasks are eligible to run. Together, these tools help balance workload distribution and ensure critical tasks are handled in an efficient way.



```
# get forex rates of JPY and push them into XCOM
get_forex_rate_USD = SimpleHttpOperator(
    task_id='get_forex_rate_USD',
    method='GET',
    pool = 'forex_api_pool'
    http_conn_id='forex_api',
    endpoint='/latest?base=USD',
    xcom_push=True
)
```

Note that it was also applied to every other task that was using the API as well.

Kubernetes Reminder

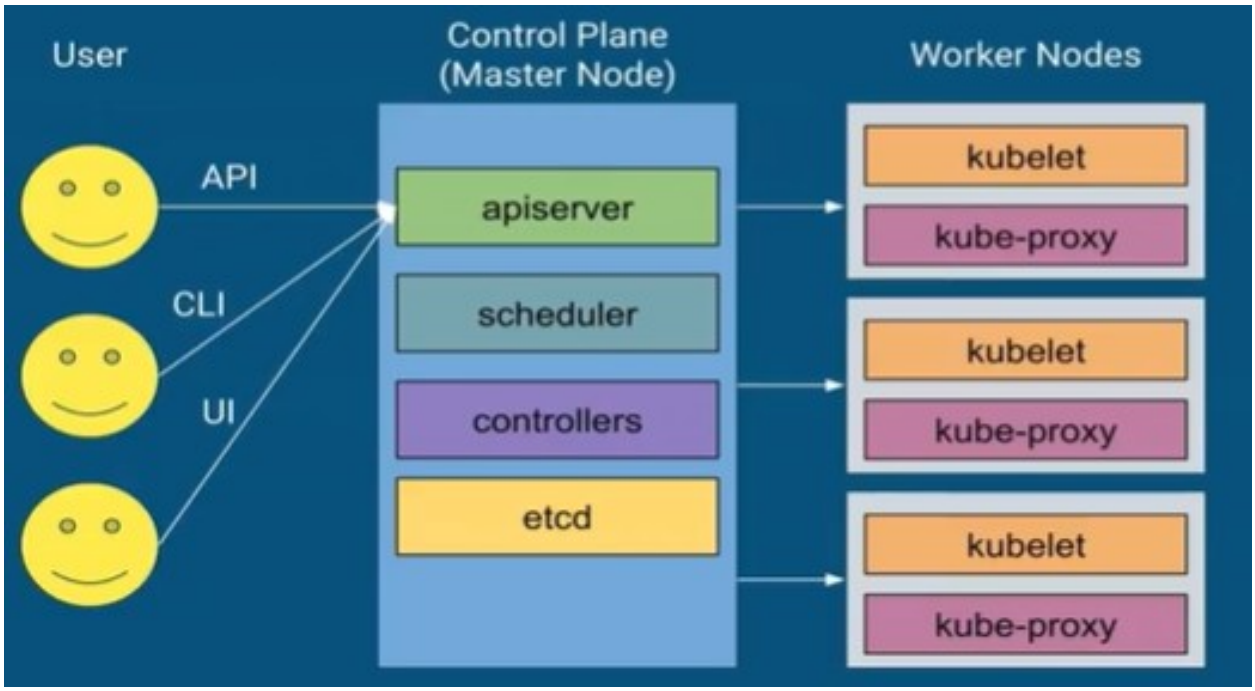
Kubernetes is extremely powerful:

- Open source project founded by Google in 2014.
- Container orchestrator.
- Kubernetes automates sysadmin tasks:
- Upgrading servers, installing security patches, configuring networks, running backups, failover, monitoring.
- Zero-downtime deployments with rolling updates.
- Provides facilities to implement CD practices: Canary / Blue-green.
- Supports autoscaling (up and down).
- Redundancy and failover built in, applications are more reliable and resilient.

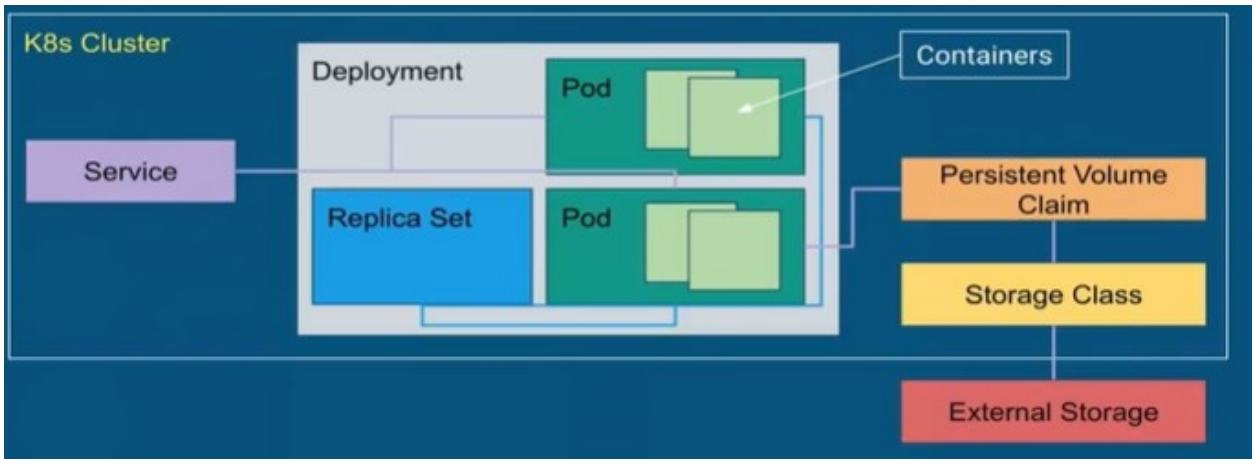
But there are some drawbacks:

- It's hard to correctly set up a K8s (kubernetes cluster).
- Hard to maintain.
- Has a steep learning curve.

Kubernetes Cluster Architecture



Important Kubernetes Objects



Scaling Airflow with Kubernetes Executors

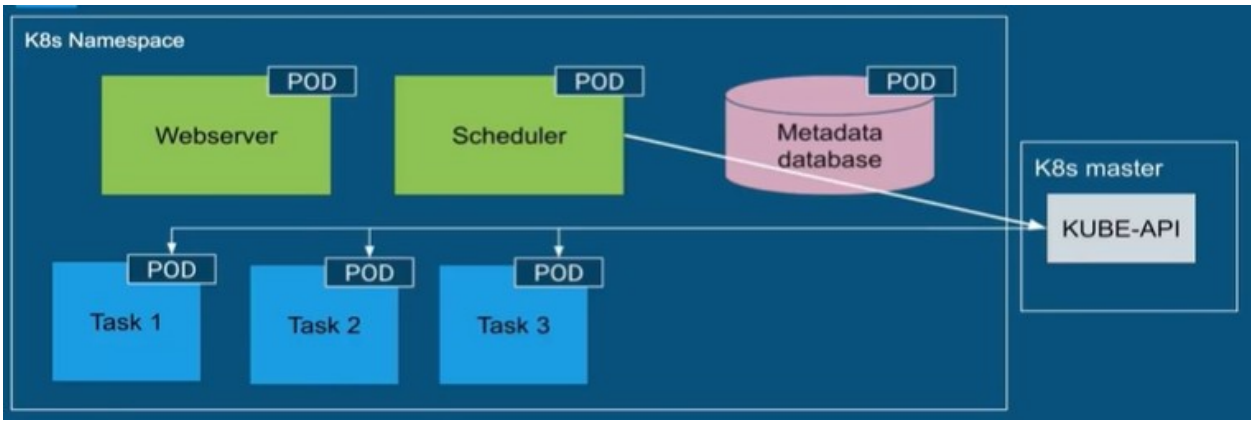
Drawbacks of the Celery Executor

- Need to set up tier applications (RabbitMQ, Celery, Flower).
- Deal with missing dependencies.
- Wasting resources: Airflow workers stay idle when no workload.
- Worker nodes are not as resilient as you think.

Nonetheless, it stays a widely used executor in production.

What is the Kubernetes Executor?

- Runs your tasks on Kubernetes.
- One task = One Pod.
- Task-level Pod configuration.
- Expands and shrinks your cluster according to the workload.
- Pods run to completion.
- Scheduler subscribes to Kubernetes event stream.
- DAG distribution:
  - Git clone with init-container for each Pod.
  - Mount volume with DAGs.
  - Build image with the DAG codes.



Setting up a 3-nodes Kubernetes Cluster with Vagrant and Rancher

Getting started with Vagrant

Firstly, it's needed to install the "vagrant-vbguest" and "vagrant-scp" plugins:

```
Marc@MacBook-Pro ~/airflow-materials/airflow-section-5/vagrant-files (master) $
 vagrant plugin install vagrant-vbguest vagrant-scp
```

Before any other commands, it's important to check if the console is being run at the directory that contains the Vagrantfile, which contains configuration arguments for it's instalation. After this is checked, the following command runs vagrant:

`vagrant up`

After it, the Virtual Machines (VMs) are already going to be running. To check it's state, run `vagrant status`:

```
Marc@MacBook-Pro ~/airflow-materials/airflow-section-5/vagrant-files (master) $
 vagrant status
Current machine states:

master                    running (virtualbox)
worker-1                  running (virtualbox)
worker-2                  running (virtualbox)

This environment represents multiple VMs. The VMs are all listed
above with their current state. For more information about a specific
VM, run `vagrant status NAME`.
```

SSH is used to connect into a node:

`vagrant ssh <node_name>`

After being logged in, the command `kubectl get nodes` is used to have an overview running nodes:

```
Marc@MacBook-Pro ~/airflow-materials/airflow-section-5/vagrant-files (master) $
 vagrant ssh master
Last login: Tue Jan 21 18:41:10 2020
[vagrant@master ~]$ kubectl get nodes
NAME                  STATUS   ROLES    AGE     VERSION
master.cluster.com    Ready    master   13m     v1.15.4
worker-1.cluster.com  Ready    <none>   8m3s    v1.15.4
worker-2.cluster.com  Ready    <none>   2m11s   v1.15.4
```

To log out, simply type `logout` when connected to the VM through SSH.

Getting started with Rancher

Rancher will be used to manage the Kubernetes cluster through a friendly interface.

To start the rancher container:

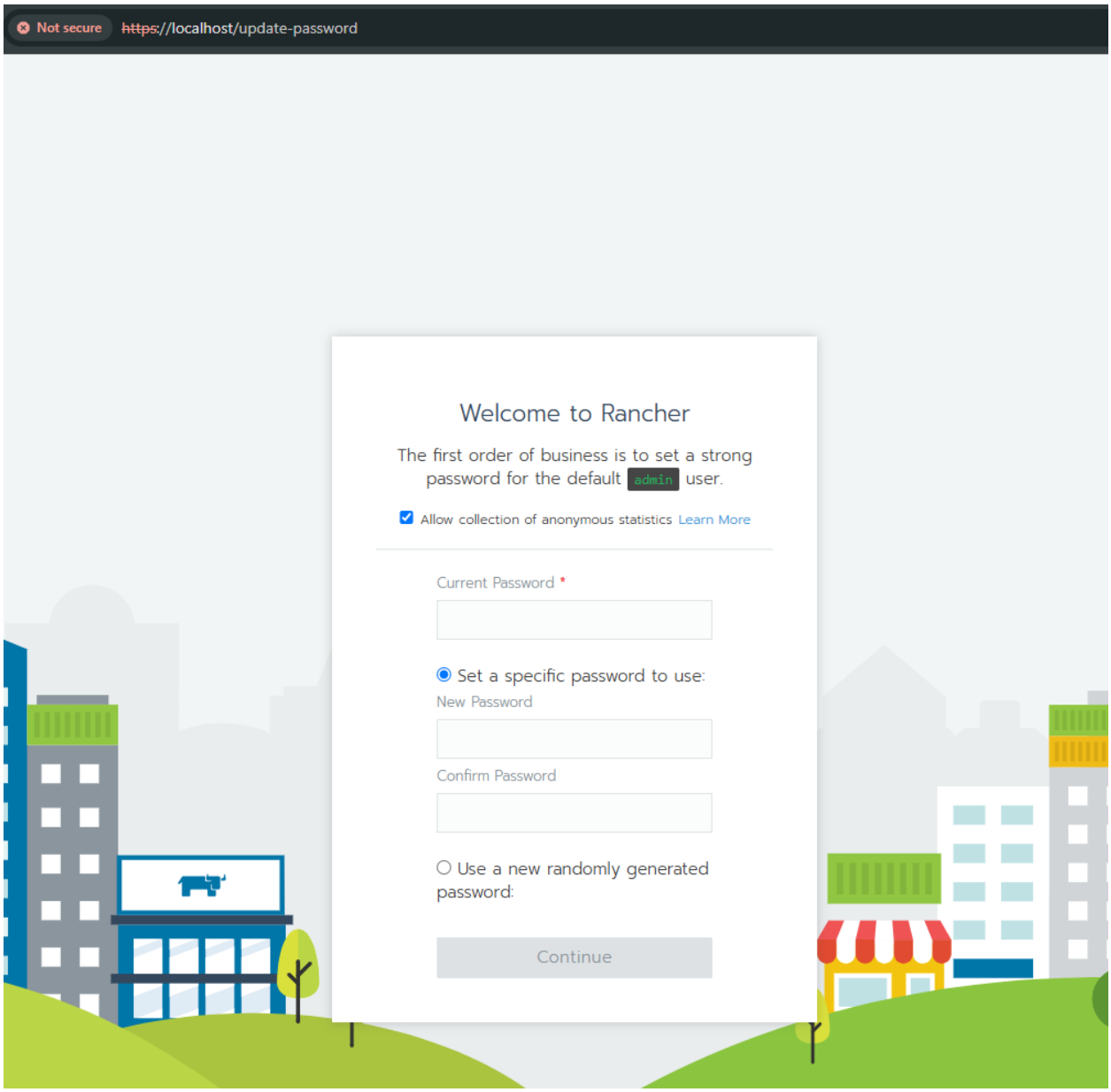
```
docker run -d --restart=unless-stopped -p 80:80 -p 443:443 --name
rancher rancher/rancher:v2.3.2
```

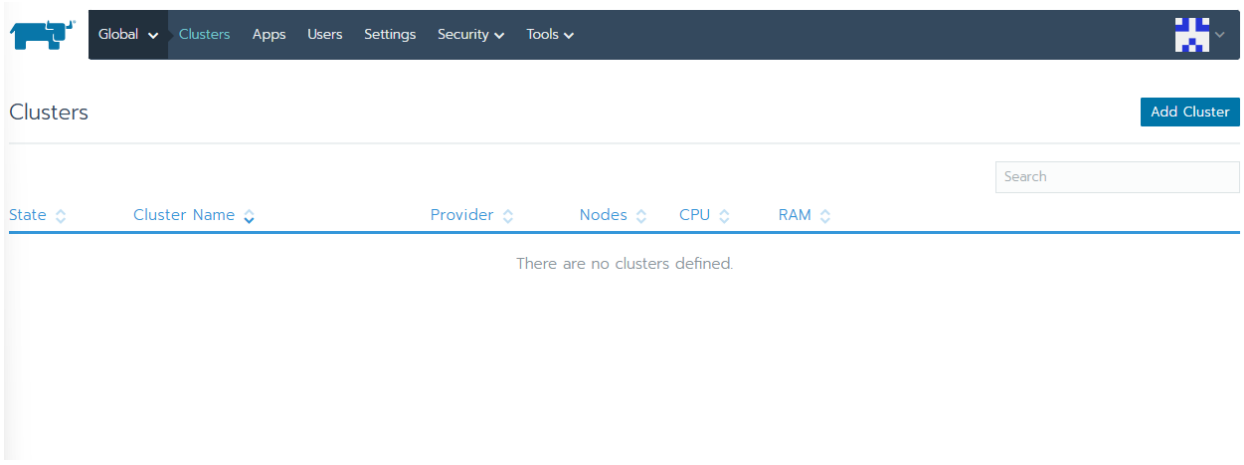
This exposes the ports 80 and 443 for it.

```
PS C:\Users\pedro> docker run -d --restart=unless-stopped -p 80:80 -p 443:443 --name rancher rancher/r
ancher:v2.3.2
Unable to find image 'rancher/rancher:v2.3.2' locally
v2.3.2: Pulling from rancher/rancher
c58094023a2e: Download complete
8a37a3d9d32f: Download complete
9acf582a7992: Download complete
11048ebae908: Download complete
322f0c253a60: Download complete
ff331cbe510b: Download complete
e1d7887879ba: Download complete
bed4e005ec0d: Download complete
22e816666fd6: Download complete
5a5441e6019b: Download complete
e403b6985877: Download complete
079b6d2a1e53: Download complete
883600f5c6cf: Download complete
74a2e9817745: Download complete
Digest: sha256:c1f125266d89ec64a60385c0d2b63869ce528881315a76387f53b87dc08815a8
Status: Downloaded newer image for rancher/rancher:v2.3.2
869031ea8452c725fa0d9cee656c7943ede5c5e17109bf89a7b4daa54d1c6c7d
PS C:\Users\pedro>
```

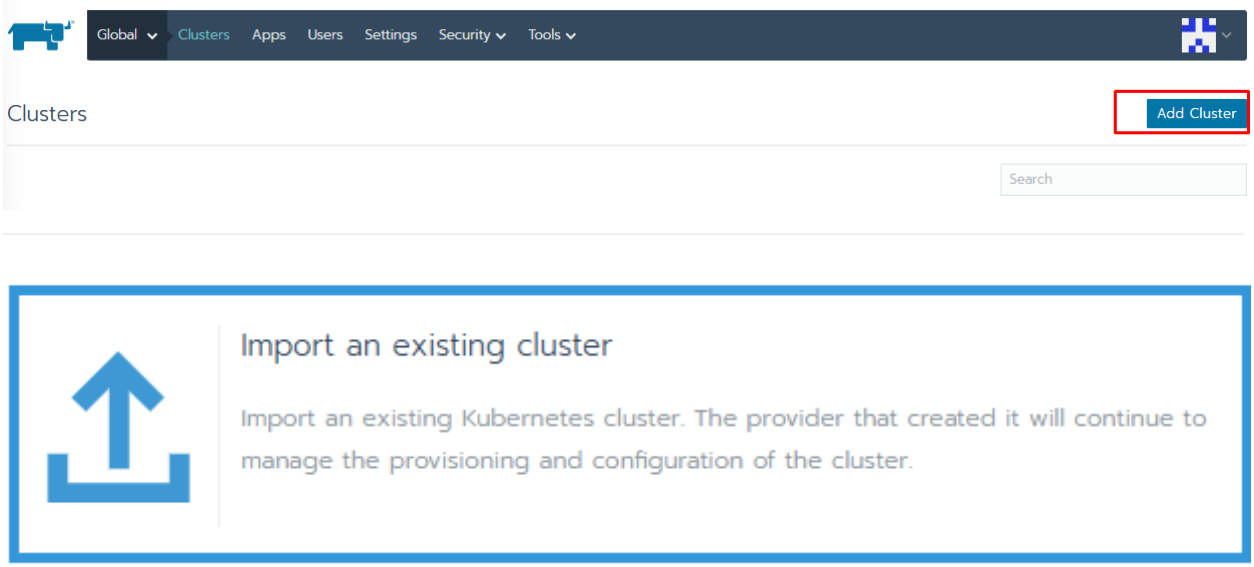
```
PS C:\Users\pedro> docker ps
CONTAINER ID   IMAGE                     COMMAND           CREATED          STATUS          PORTS
                            NAMES
869031ea8452   rancher/rancher:v2.3.2   "entrypoint.sh"   30 seconds ago   Up 29 seconds   0.0.0.0:80-
>80/tcp, 0.0.0.0:443->443/tcp   rancher
PS C:\Users\pedro>
```

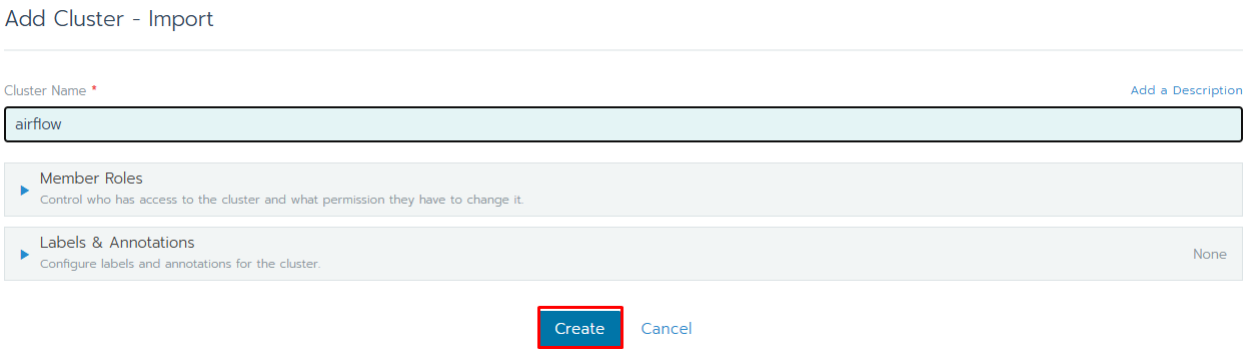
Now it can be accessed through the web browser, at localhost in the specified ports:


Not secure
https://localhost/update-password
Welcome to Rancher
The first order of business is to set a strong password for the default admin user.
Allow collection of anonymous statistics Learn More
Current Password *
Set a specific password to use:
New Password
Confirm Password
Use a new randomly generated password:
Continue

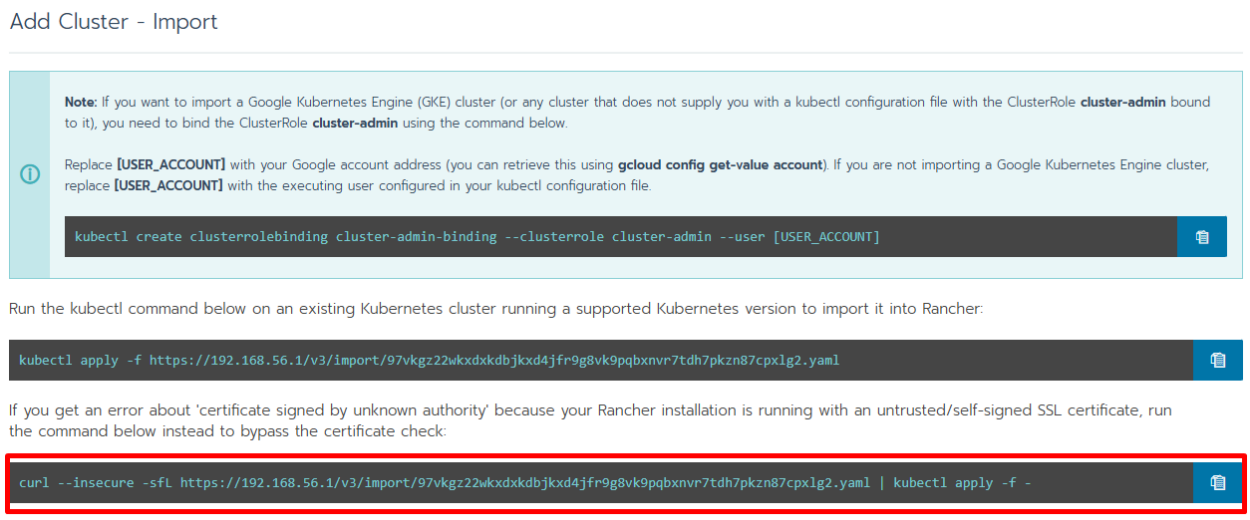

Global
Clusters
Apps
Users
Settings
Security
Tools
Clusters
Add Cluster
Search
State
Cluster Name
Provider
Nodes
CPU
RAM
There are no clusters defined.

Adding a new cluster



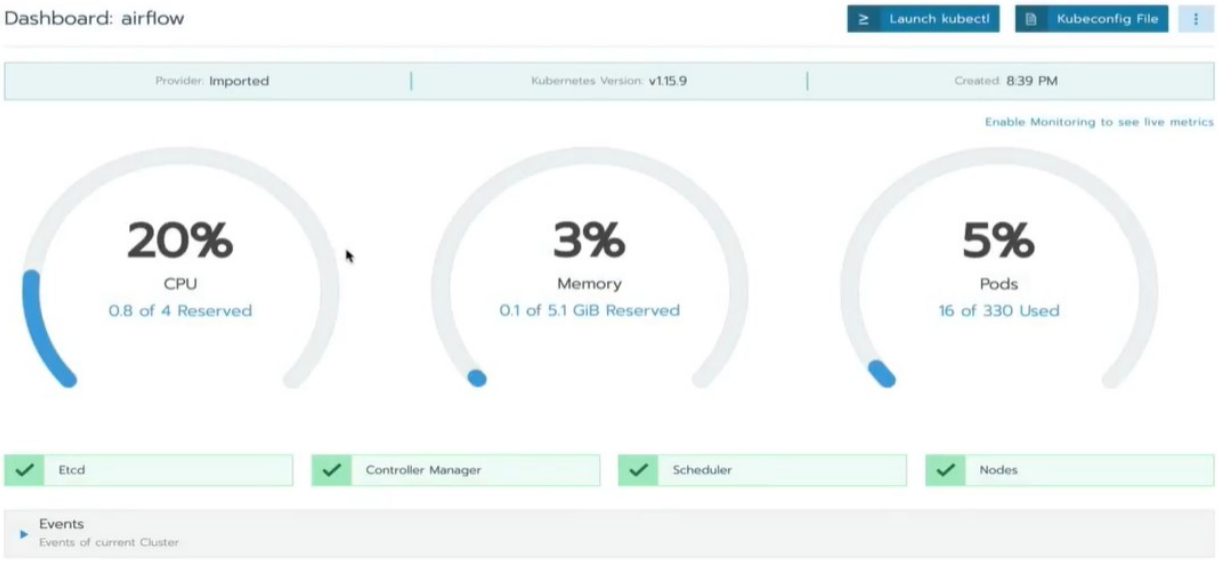


Now, the following command needs to be executed in the master node in order to import the cluster into Rancher:



After it, it should have been sucessfully imported into Rancher:

Now, system resources can be monitored through Rancher's dashboard:



Improving your DAG with advanced concepts

In the case of many parallels tasks, they can be grouped up in a single task, using the SubDagOperator, which calls the Factory Function. Note that this does not alter any dag configuration, its used only for better visual clarity for diagrams.

Practicing

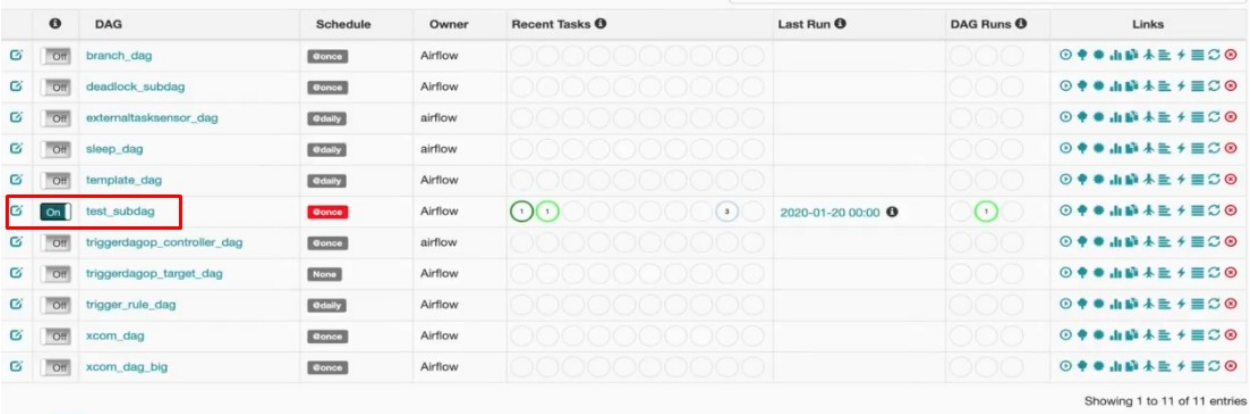
Firstly, executing airlfow:

```
Marc@macbook-pro ~/airflow-materials/airflow-section-6 (master) $ pwd
/Users/Marc/airflow-materials/airflow-section-6
Marc@macbook-pro ~/airflow-materials/airflow-section-6 (master) $ docker-compos
e -f docker-compose-CeleryExecutor.yml up -d
Creating network "airflow-section-6_default" with the default driver
Creating airflow-section-6_postgres_1 ... done
Creating airflow-section-6_redis_1    ... done
Creating airflow-section-6_flower_1   ... done
Creating airflow-section-6_webserver_1 ... done
Creating airflow-section-6_scheduler_1 ... done
Creating airflow-section-6_worker_1   ...
```

```
DAG_NAME="test_subdag"

default_args = {
    'owner': 'Airflow',
    'start_date': airflow.utils.dates.days_ago(2)
}

with DAG(dag_id=DAG_NAME, default_args=default_args, schedule_interval="@once") as dag:
    start = DummyOperator(
        task_id='start'
    )

    subdag_1 = SubDagOperator(
        task_id='subdag-1',
        subdag=factory_subdag(DAG_NAME, 'subdag-1', default_args),
        executor=SequentialExecutor()
    )

    some_other_task = DummyOperator(
        task_id='check'
        )

    subdag_2 = SubDagOperator(
        task_id='subdag-2',
        subdag=factory_subdag(DAG_NAME, 'subdag-2', default_args),
        executor=SequentialExecutor()
    )

    end = DummyOperator(
        task_id='final'
    )

    start >> subdag_1 >> some_other_task >> subdag_2 >> end
```

*Note that the subdags can run in parallel if the operator is changed from SequentialOperator to CeleryExecutor.

Now, executing the task and waiting for it to be completed:

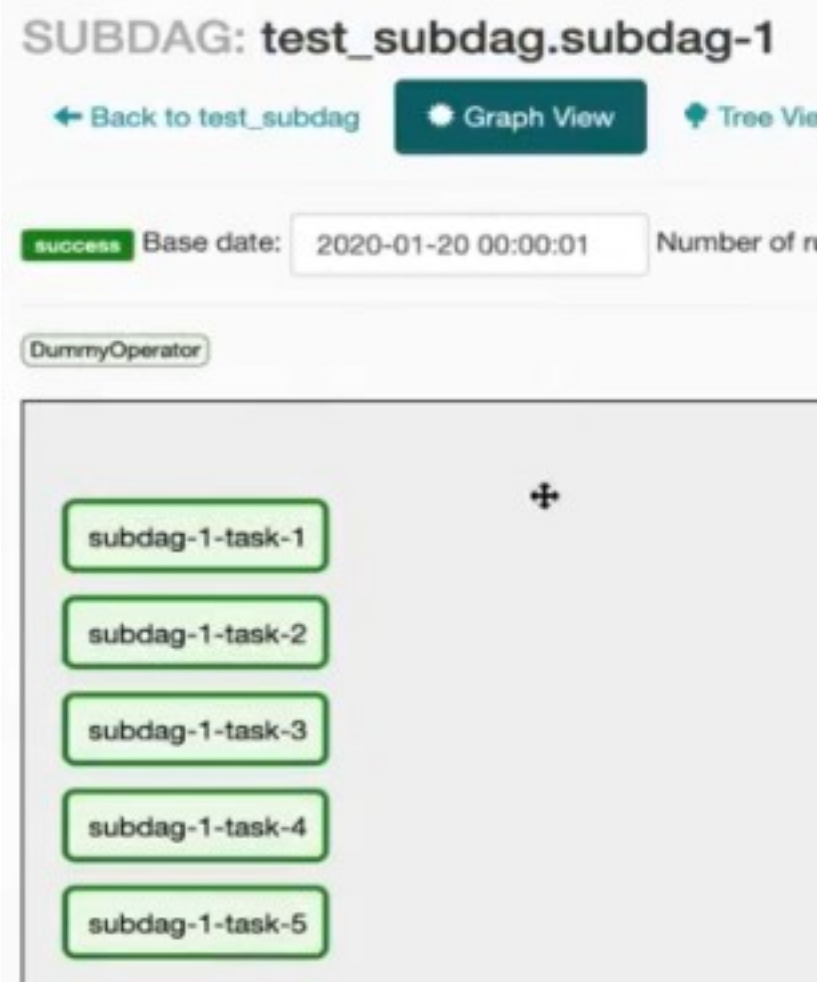


	DAG	Schedule	Owner	Recent Tasks	Last Run	DAG Runs	Links
Off	branch_dag	@once	Airflow				
Off	deadlock_subdag	@once	Airflow				
Off	externaltasksensor_dag	@daily	airflow				
Off	sleep_dag	@daily	airflow				
Off	template_dag	@daily	Airflow				
On	test_subdag	@once	Airflow	1 1 3	2020-01-20 00:00	1	
Off	triggerdagop_controller_dag	@once	airflow				
Off	triggerdagop_target_dag	None	Airflow				
Off	trigger_rule_dag	@daily	Airflow				
Off	xcom_dag	@once	Airflow				
Off	xcom_dag_big	@once	Airflow				

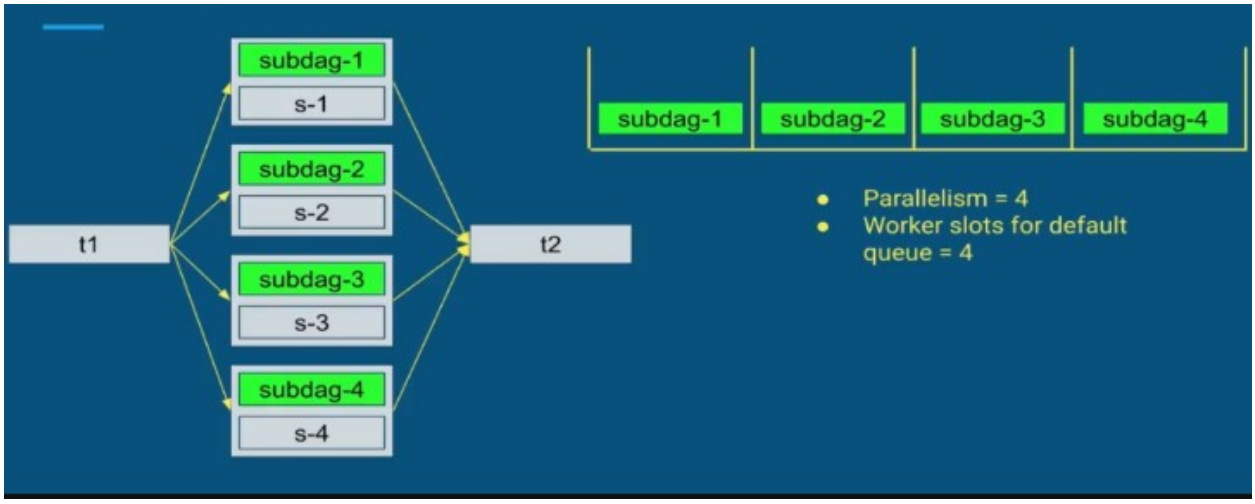
Showing 1 to 11 of 11 entries

Now, the subtask has been already created:

And by clicking on it, its possible to visualize all the tasks included in it:



SubDag DeadLock



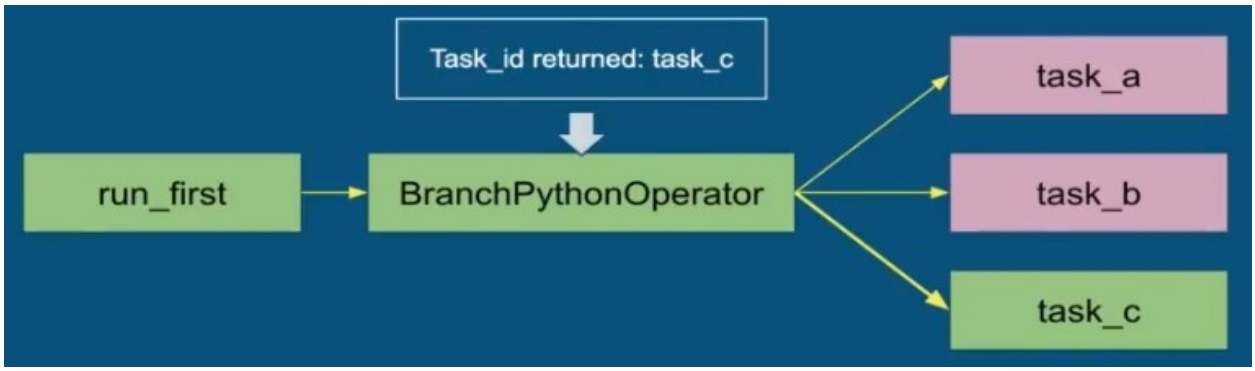
DeadLock happens because each SubDag is treated as a task, so when parallelism is set to 4, there are 4 subdags to be run and each subdag contains a task, the subdags will take all the slots for parallelism, making the execution of its tasks not possible, creating a deadlock.

A queue can be used to fix it, but to make things easy, just simply use the sequential executor or simply not use subdags at all.

Making different paths in your DAGs with Branching

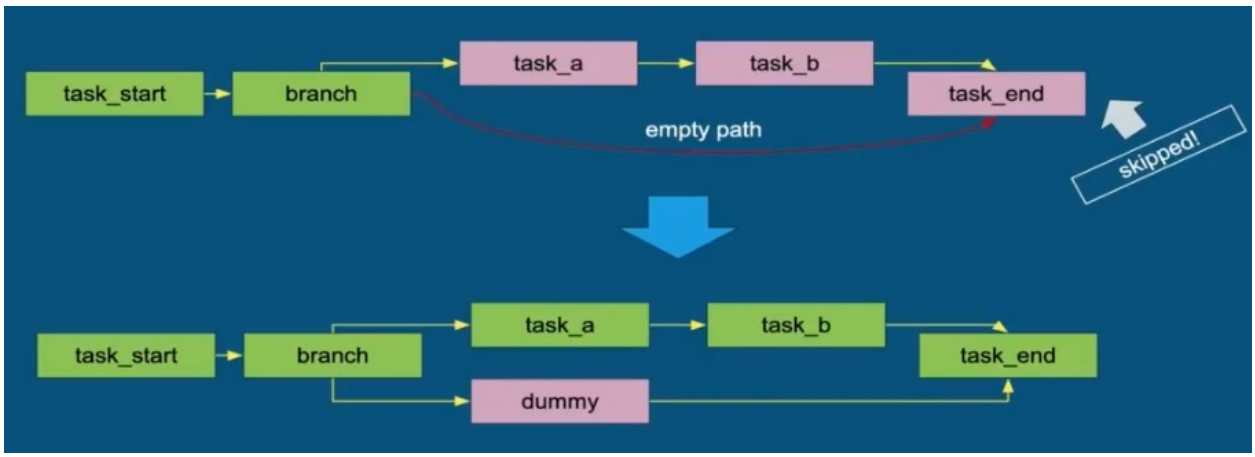
Branching is the mechanism that allows your DAG to choose between different paths according to the result of a specific task. Can be referenced as BranchPythonOperator.

It works by returning a task_id, which will be used to execute an specific task after its completed:



Important Notes

It's not recommended to be skipping tasks, so if at some point it turns out to be useful, the dag needs to have another path to get into the end of the dag run. It can be done by creating a dummy task directly connected to the end of the task by using the branch operator:

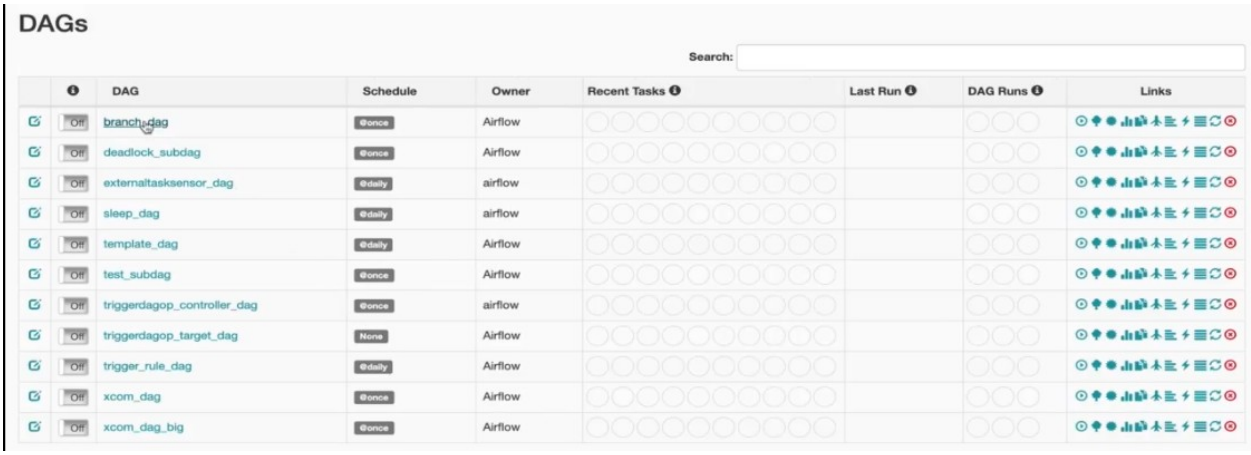


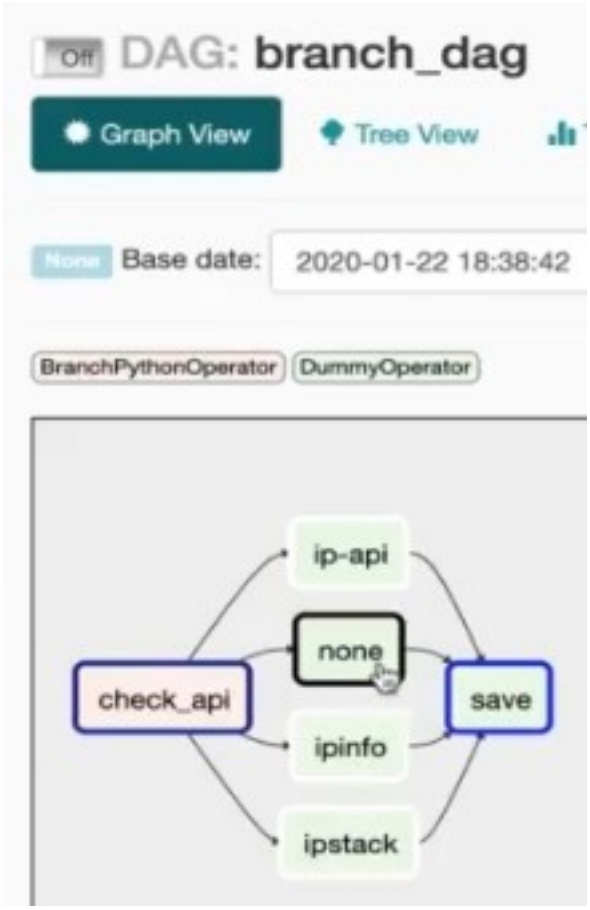
Practicing

branch_dag.py:

```python
IP_GEOLOCATION_APIS = {
    'ip-api': 'http://ip-api.com/json/',
    'ipstack': 'https://api.ipstack.com/',
    'ipinfo': 'https://ipinfo.io/json'
}

# Try to get the country_code field from each API
# If given, the API is returned and the next task corresponding
# to this API will be executed
def check_api():
    for api, link in IP_GEOLOCATION_APIS.items():
        r = requests.get(link)
        try:
            data = r.json()
            if data and 'country' in data and len(data['country']):
                return api
        except ValueError:
            pass
    return 'none'

with DAG(dag_id='branch_dag',
    default_args=default_args,
    schedule_interval="@once") as dag:

    # BranchPythonOperator
    # The next task depends on the return from the
    # python function check_api
    check_api = BranchPythonOperator(
        task_id='check_api',
        python_callable=check_api
    )

    none = DummyOperator(
        task_id='none'
    )

    save = DummyOperator(task_id='save')

    check_api >> none >> save
```

Clicking on this task in airflow:





After the task is completed, it's possible to visualize that the only task that has been executed was the ip-api, which was returned by the branch operator:

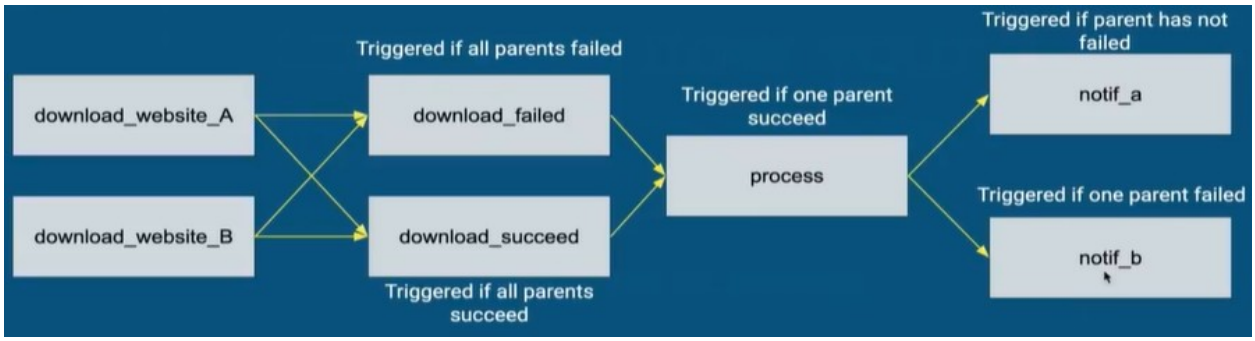
```
.py:105} INFO - Exporting the following env vars:

):00+00:00
20T00:00:00+00:00
.py:114} INFO - Done. Returned value was: ip-api
} INFO - Following branch ip-api
```

Trigger rules for your tasks

Trigger rules makes possible to decide which task will be executed based on the result of the previous task: if it was successful or not (all_success / all_failed); if at least one of the previous tasks were not failed (none_failed) and many other:

Practicing

For this practice, the following use case will be implemented:



Here, the code for the tasks, which shows what are the trigger rules for each of the tasks present in the dag.

```python
with DAG(dag_id='trigger_rule_dag',
    default_args=default_args,
    schedule_interval="@daily") as dag:

    # Change the trigger rules according to the
    # dag from the video
    # all_success
    # all_failed
    # all_done
    # one_failed
    # one_success
    # none_failed
    # none_skipped

    download_website_a_task = PythonOperator(
        task_id='download_website_a',
        python_callable=download_website_a,
        trigger_rule="all_success"
    )

    download_website_b_task = PythonOperator(
        task_id='download_website_b',
        python_callable=download_website_b,
        trigger_rule="all_success"
    )

    download_failed_task = PythonOperator(
        task_id='download_failed',
        python_callable=download_failed,
        trigger_rule="all_success"
    )

    download_succeed_task = PythonOperator(
        task_id='download_succeed',
        python_callable=download_succeed,
        trigger_rule="all_success"
    )

    process_task = PythonOperator(
        task_id='process',
        python_callable=process,
        trigger_rule="all_success"
    )

    notif_a_task = PythonOperator(
        task_id='notif_a',
        python_callable=notif_a,
        trigger_rule="all_success"
```

On airflow:

As it can be seen, there is no dependecies yet between each tasks. For this, some modifications needs to be taken into the python dag file:

```
101
102     [ download_failed, download_succeed] >> process_task
```

Now, the applied dependencies can be seen in the graph view:

Setting up the dependencies to fit the use case above:

```
96    # Implement dependencies below
97    # ie:
98    # a >> b        : b depends on a
99    # [a b] >> c    : c depends on a and b
100   # ...
101
102   [ download_failed, download_succeed] >> process_task
103   [ download_website_a_task, download_website_b_task] >> download_failed_task
104   [ download_website_a_task, download_website_b_task] >> download_succeed_task
105   process_task >> [ notif_a_task, notif_b_task ]
```

Now, the graph view should be the same as the use case:

Now that the dependencies are all set, its time to set the correct trigger rules for each task.

Setting up the trigger rules correctly

1. `download_failed_task` needs to have it trigger rule set to "all_failed".

```
download_failed_task = PythonOperator(
    task_id='download_failed',
    python_callable=download_failed,
    trigger_rule="all_failed"
)
```

1. `process_task` needs to have it trigger rule set to "one_sucess", since the task will either fail or succeed.

```
process_task = PythonOperator(
    task_id='process',
    python_callable=process,
    trigger_rule="one_success"
)
```

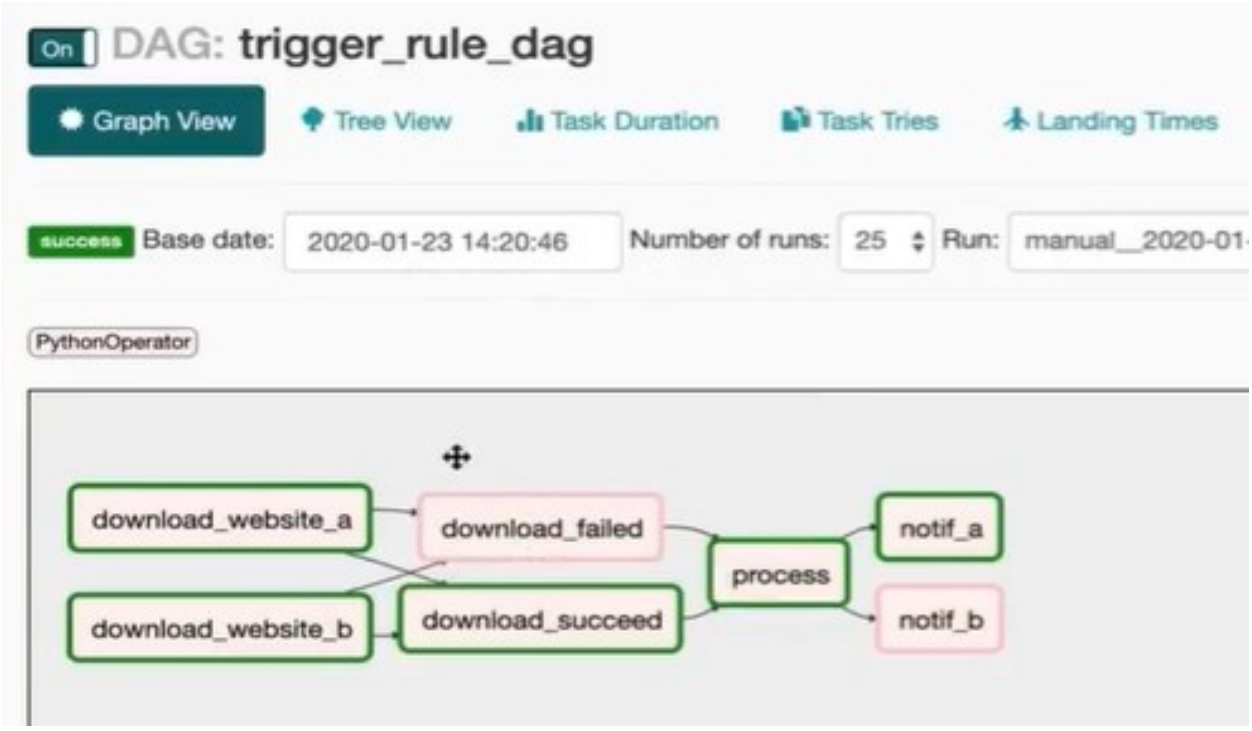
1. `notif_a_task` needs to have it trigger rule set to "none_failed".

```
notif_a_task = PythonOperator(
    task_id='notif_a',
    python_callable=notif_a,
    trigger_rule="none_failed"
)
```

1. `notif_b_task` must be run as soon as at least one parent has failed, so the trigger rule should be set to "one_failed".

```
notif_b_task = PythonOperator(
    task_id='notif_b',
    python_callable=notif_b,
    trigger_rule="one_failed"
)
```

With that changed being appiled, lets now trigger the dag:



Since both download tasks were successful, download_failed was skipped, the same applies to notif_b, since nothing has failed.

Templating your tasks

Templating the tasks are important to maintain consistent information throughout the output obtained through the execution of a dag in different instances in time.

```
with DAG(dag_id="template_dag", schedule_interval="@daily", default_args=default_args) as dag:

    t0 = BashOperator(
            task_id="t0",
            bash_command="echo {{ macros.ds_format(ts_nodash, '%Y%m%dT%H%M%S', '%Y-%m-%d-%H-%M') }}")

    t1 = BashOperator(
            task_id="generate_new_logs",
            bash_command="./scripts/generate_new_logs.sh",
            params={'filename': 'log.csv'})

    t2 = BashOperator(
            task_id="logs_exist",
            bash_command="test -f " + TEMPLATED_LOG_DIR + "log.csv",
            )

    t3 = PythonOperator(
            task_id="process_logs",
            python_callable=process_logs_func,
            provide_context=True,
            templates_dict={'log_dir': TEMPLATED_LOG_DIR},
            params={'filename': 'log.csv'}
            )

    t0 >> t1 >> t2 >> t3
```

The task t0 shows the current date of execution, formatting it using a macro.

The task t1 shows executes the script "generete_new_logs.sh" and saves its output in a csv file.

The task t2 checks if a file named log.csv exists in the directory represented by the variable.

The task t3 saves the csv file into the system.

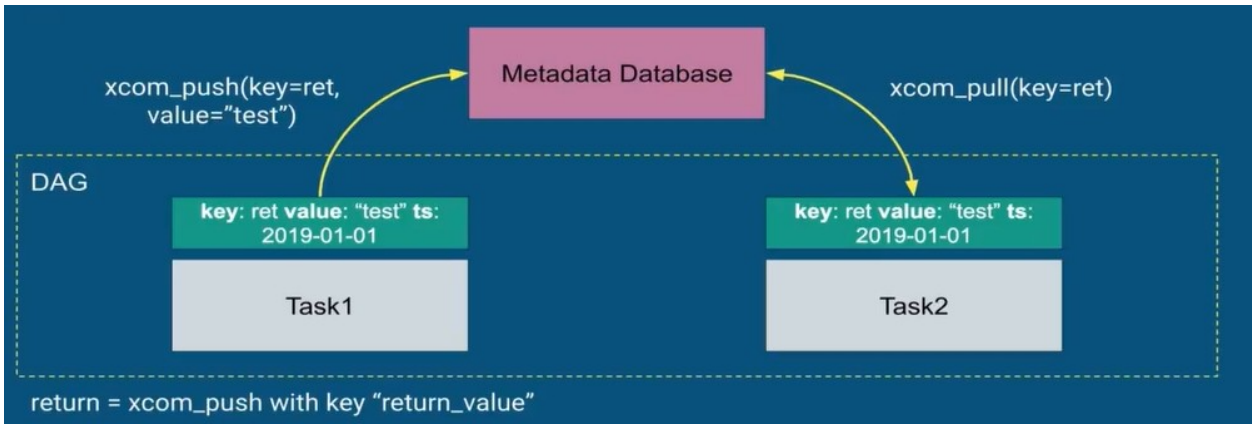
With this, data can now have more consistency, setting new variables accordingly to dynamic dates, useful for regular SQL searches, for example.

Sharing data between tasks with XCOM

XCOM allows multiple tasks to exchange messages between them, using keys, values and timestamps.

To push some data into the metadatabase, the function `xcom_push(key=ret, value="test")` can be used (timestamp is the date the data was pushed into the database).

After a task has pushed data into the database, another task can access the data using the function `xcom_pull(key=ret)`, specifying the key of the desired data:


Metadata Database
xcom_push(key=ret, value="test")
xcom_pull(key=ret)
DAG
key: ret value: "test" ts: 2019-01-01
key: ret value: "test" ts: 2019-01-01
Task1
Task2
return = xcom_push with key "return_value"

Practicing

xcom_dag.py 3

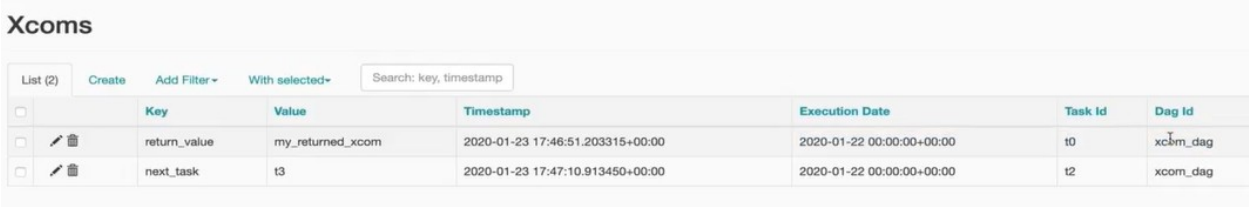
airflow-section-6 > mnt > airflow > dags > xcom_dag.py > get_next_task

```python

args = {
    'owner': 'Airflow',
    'start_date': airflow.utils.dates.days_ago(1),
}

def push_xcom_with_return():
    return 'my_returned_xcom'

def get_pushed_xcom_with_return(**context):
    print(context['ti'].xcom_pull(task_ids='t0'))

def push_next_task(**context):
    context['ti'].xcom_push(key='next_task', value='t3')

def get_next_task(**context):
    return context['ti'].xcom_pull(key='next_task')

def get_multiple_xcoms(**context):
    print(context['ti'].xcom_pull(key=None, task_ids=['t0', 't2']))

with DAG(dag_id='xcom_dag', default_args=args, schedule_interval="@once") as dag:

    t0 = PythonOperator(
        task_id='t0',
        python_callable=push_xcom_with_return
    )

    t1 = PythonOperator(
        task_id='t1',
        provide_context=True,
        python_callable=get_pushed_xcom_with_return
    )

    t2 = PythonOperator(
        task_id='t2',
        provide_context=True,
        python_callable=push_next_task
    )

    branching = BranchPythonOperator(
        task_id='branching',
        provide_context=True,
        python_callable=get_next_task,
    )

    t3 = DummyOperator(task_id='t3')
```

```python
t4 = DummyOperator(task_id='t4')

t5 = PythonOperator(
    task_id='t5',
    trigger_rule='one_success',
    provide_context=True,
    python_callable=get_multiple_xcoms
)

t6 = BashOperator(
    task_id='t6',
    bash_command="echo value from xcom: {{ ti.xcom_pull(key='next_task') }}"
)

t0 >> t1
t1 >> t2 >> branching
branching >> t3 >> t5 >> t6
branching >> t4 >> t5 >> t6
```

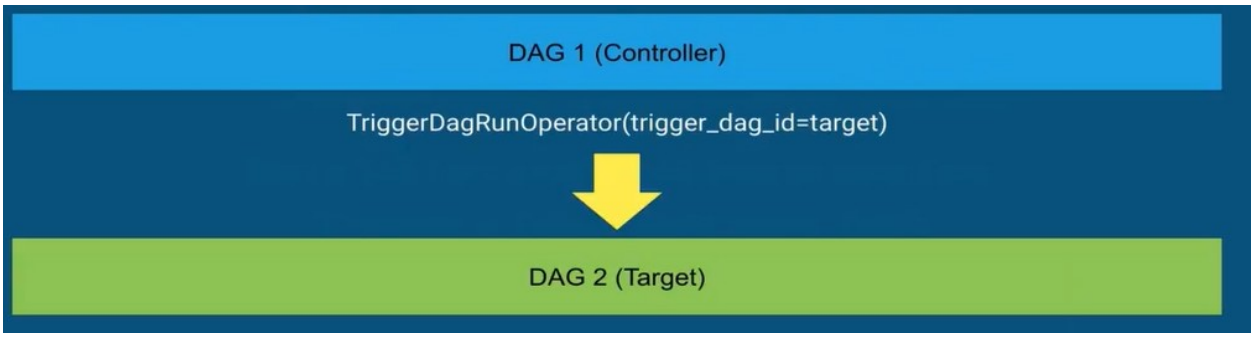
In this dag, there are some tasks that push/pull data to/from the database, as well a branch operator, to decide which task will be executed based on the data pulled using xcom.

By triggering this dag, some xcoms are pushed:



TriggerDagRunOperator or when your DAG controls another DAG

Its used to trigger a dag from another dag. Its usefuL to modularize tasks execution so a DAG doesnt contain an insane amount of tasks, which would make its execution confusing and hard to maintain. Checking the following use case:



TriggerDagRunOperator parameters:

- trigger_dag_id
- pyhton_callable
- params

Important notes:

- Controller does not wait for target to finish
- Controller and target are independent
- No visualization from the controller neither history
- Both DAGs must be scheduled
- Target DAG schedule interval = None

Practicing

Below, there are the files for the dag configuration for the controller and target dag.

triggerdagop_controller_dag.py 2 | triggerdagop_target_dag.py 2

airflow-section-6 > mnt > airflow > dags > triggerdagop_controller_dag.py > ...

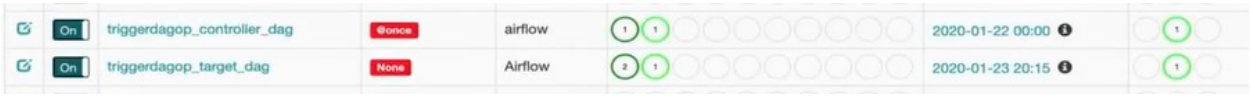
```python
import pprint as pp
import airflow.utils.dates
from airflow import DAG
from airflow.operators.dagrun_operator import TriggerDagRunOperator
from airflow.operators.dummy_operator import DummyOperator

default_args = {
        "owner": "airflow",
        "start_date": airflow.utils.dates.days_ago(1)
    }

def conditionally_trigger(context, dag_run_obj):
    if context['params']['condition_param']:
        dag_run_obj.payload = {
                'message': context['params']['message']
            }
        pp.pprint(dag_run_obj.payload)
        return dag_run_obj

with DAG(dag_id="triggerdagop_controller_dag", default_args=default_args, schedule_interval="@once") as dag:
    trigger = TriggerDagRunOperator(
        task_id="trigger_dag",
        trigger_dag_id="triggerdagop_target_dag",
        provide_context=True,
        python_callable=conditionally_trigger,
        params={
            'condition_param': True,
            'message': 'Hi from the controller'
        },
    )

    last_task = DummyOperator(task_id="last_task")

    trigger >> last_task
```

triggerdagop_controller_dag.py 2 | triggerdagop_target_dag.py 2

airflow-section-6 > mnt > airflow > dags > triggerdagop_target_dag.py > ...

```python
import airflow.utils.dates
from airflow.models import DAG
from airflow.operators.bash_operator import BashOperator
from airflow.operators.python_operator import PythonOperator

default_args = {
    "start_date": airflow.utils.dates.days_ago(1),
    "owner": "Airflow"
}

def remote_value(**context):
    print("Value {} for key=message received from the controller DAG".format(context["dag_run"].conf["message"]))

with DAG(dag_id="triggerdagop_target_dag", default_args=default_args, schedule_interval=None) as dag:

    t1 = PythonOperator(
            task_id="t1",
            provide_context=True,
            python_callable=remote_value,
        )

    t2 = BashOperator(
        task_id="t2",
        bash_command='echo Message: {{ dag_run.conf["message"] if dag_run else "" }}')

    t3 = BashOperator(
        task_id="t3",
        bash_command="sleep 30"
    )
```

Triggering the controller and targetg dag and watching what happens to the target dag:



Looking at the logs in the target dag to comprehend what's happened:

For the task t1, it completes its execution by showing the information passed from the controller dag:

```
[2020-01-23 20:16:05,575] {python_operator.py:105} INFO - Exporting the following env vars:
AIRFLOW_CTX_DAG_ID=triggerdagop_target_dag
AIRFLOW_CTX_TASK_ID=t1
AIRFLOW_CTX_EXECUTION_DATE=2020-01-23T20:15:58.064183+00:00
AIRFLOW_CTX_DAG_RUN_ID=trig__2020-01-23T20:15:58.030014+00:00
[2020-01-23 20:16:05,575] {logging_mixin.py:95} INFO - Value Hi from the controller for key=message received from the controller DAG
[2020-01-23 20:16:05,575] {python_operator.py:114} INFO - Done. Returned value was: None
[2020-01-23 20:16:07,744] {logging_mixin.py:95} INFO - [ [34m2020-01-23 20:16:07,743 [0m] { [34mlocal_task_job.py: [0m105} INFO [0m - Task e
```

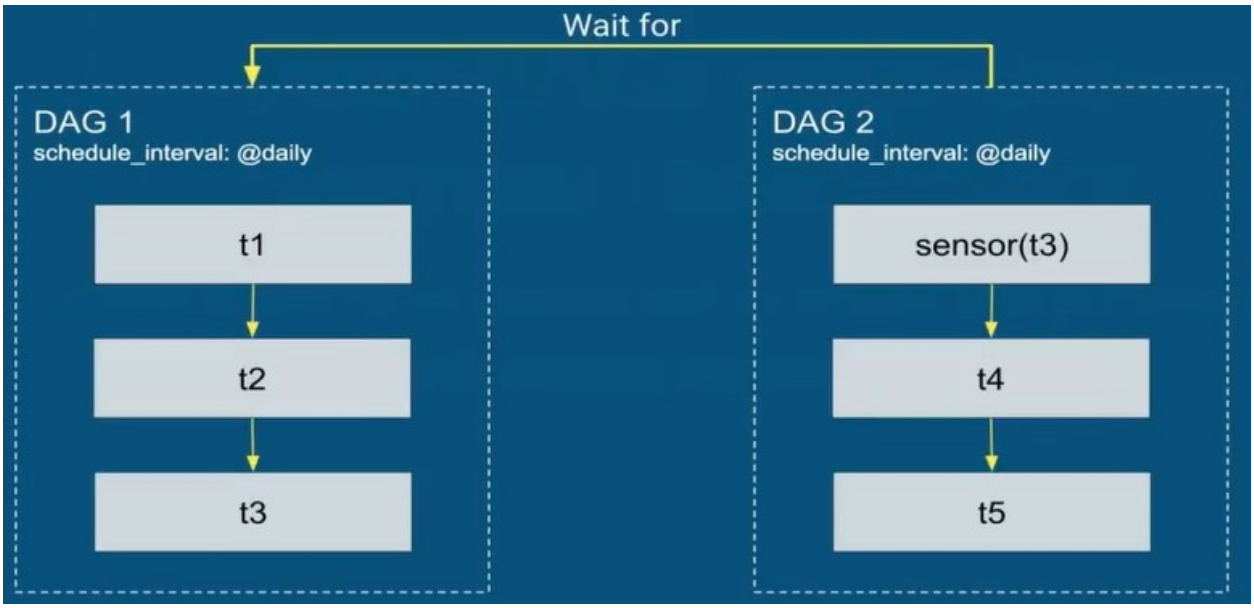
For the task t2, the same happens:

```
AIRFLOW_CTX_EXECUTION_DATE=2020-01-23T20:15:58.064183+00:00
AIRFLOW_CTX_DAG_RUN_ID=trig__2020-01-23T20:15:58.030014+00:00
[2020-01-23 20:16:05,553] {bash_operator.py:105} INFO - Temporary script location: /tmp/airflowtmp_a111dyz/t2yy870r8o
[2020-01-23 20:16:05,553] {bash_operator.py:115} INFO - Running command: echo Message: Hi from the controller
[2020-01-23 20:16:05,566] {bash_operator.py:124} INFO - Output:
[2020-01-23 20:16:05,568] {bash_operator.py:128} INFO - Message: Hi from the controller
[2020-01-23 20:16:05,568] {bash_operator.py:132} INFO - Command exited with return code 0
[2020-01-23 20:16:07,836] {logging_mixin.py:95} INFO - [ [34m2020-01-23 20:16:07,836 [0m] { [34mlocal_task_job.py: [0m105} INF
```

Thus, making the communication between DAGs successful.

Dependencies between your dags with the ExternalTaskSensor

It allows to wait for an exeternal task (in another dag) to be completed.



So the DAG 2 will only start running its first task when the DAG 1 has already completed all of its tasks.

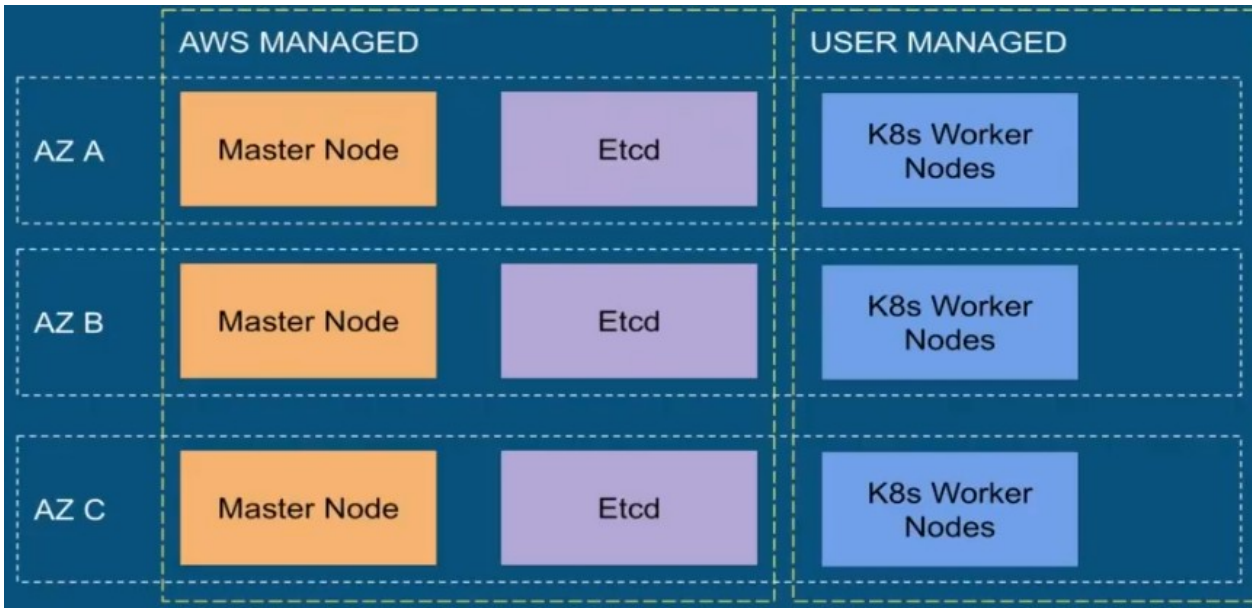
Important Notes

- Keep the same schedule between your DAGs
- Or use execution_delta or execution_date_fn parameters
- ExternalTaskSensor is different from TriggerDagRunOperator
  - Using both usually leads to getting the sensor stuck
- Mode "poke" used by default
  - "reschedule" if needed

Deploying Airflow on AWS EKS with Kubernetes Executors and Rancher

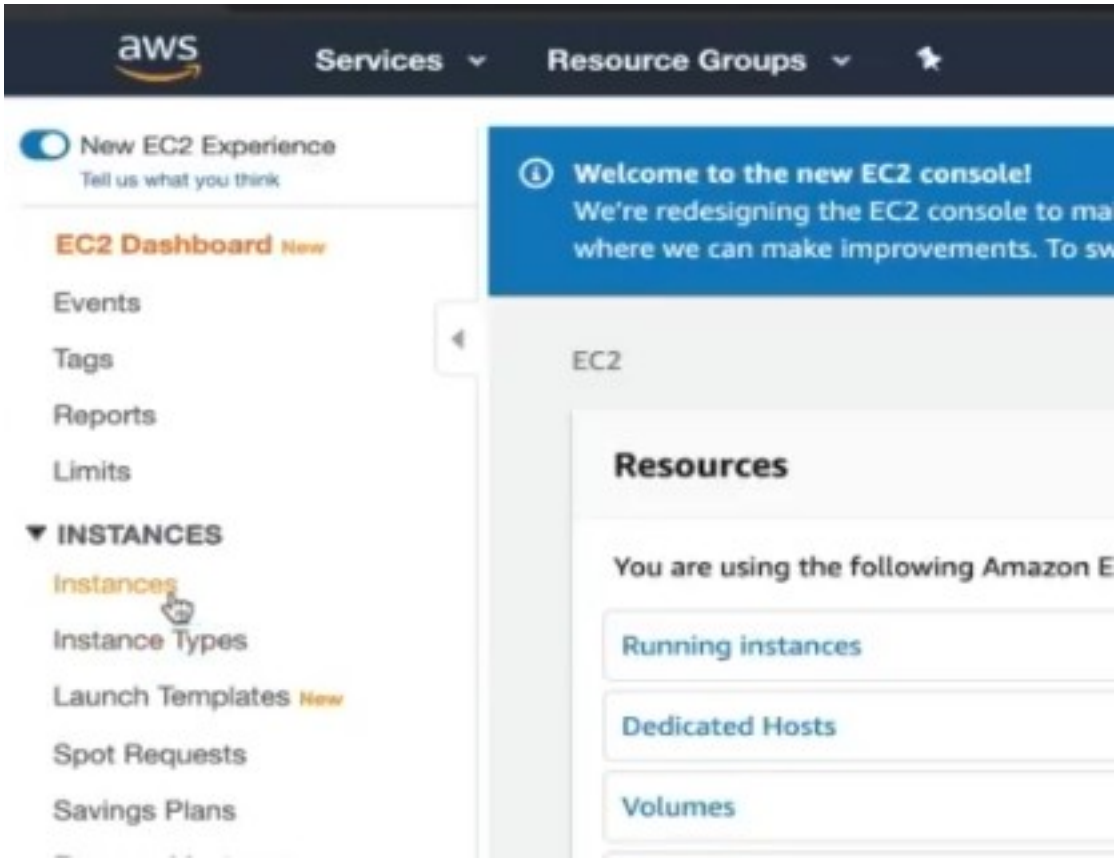
Overview:

- AWS EKS (Elastic Kubernetes Service)
- Makes it easy to deploy, manage, and scale containerized applications
- Orchestrate your containers
- Still need to configure subnets, VPS, IAM roles, SSH keys...
- Integrated with AWS services (ECR, ELB, IAM, ...)

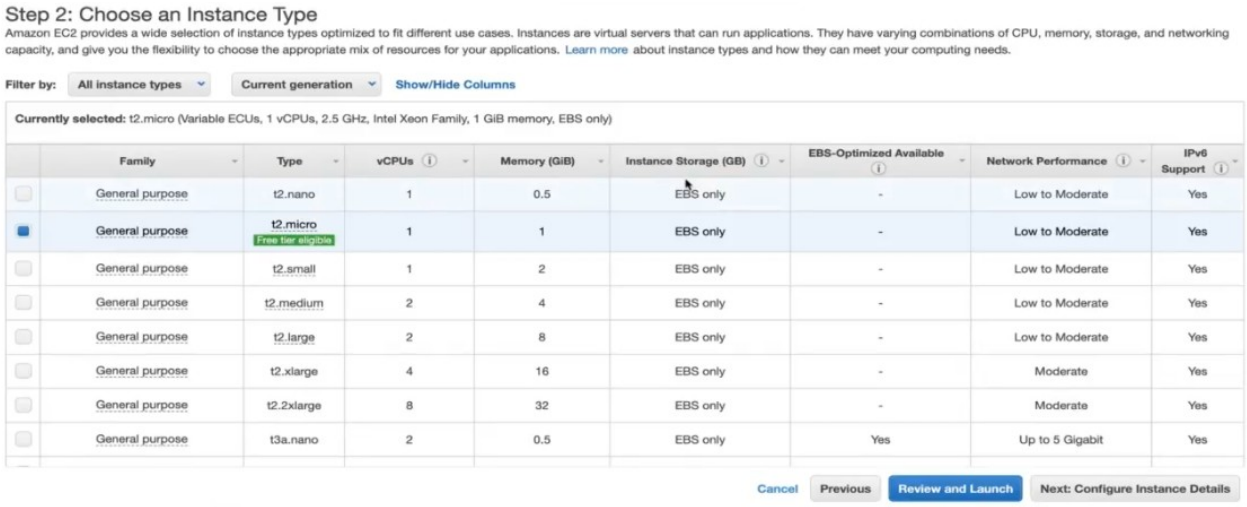


Setting up an EC2 instance for Rancher

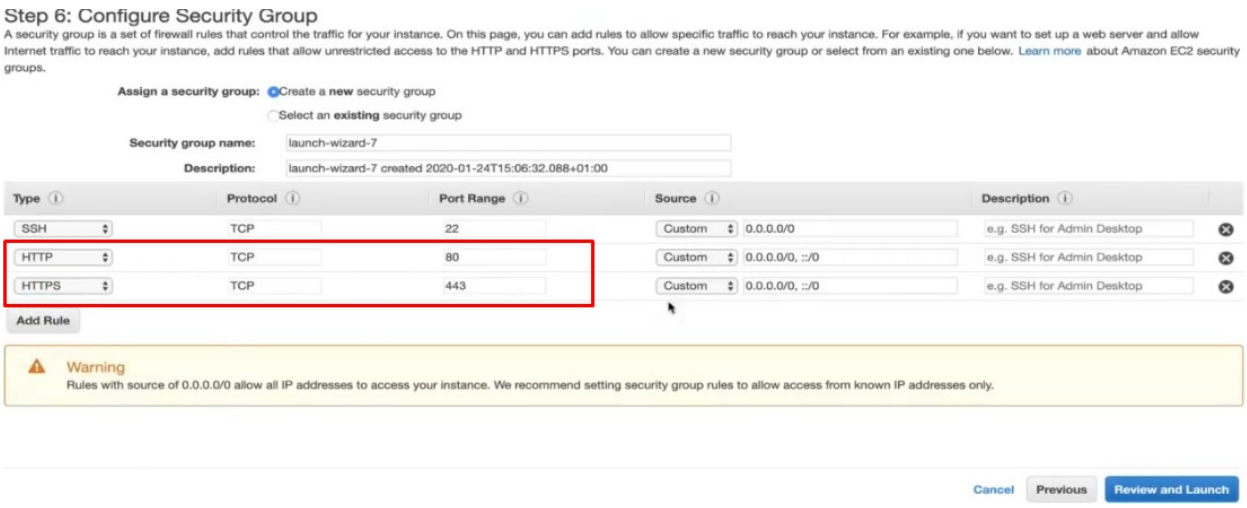
Once logged in the AWS console, click on "instances" and then



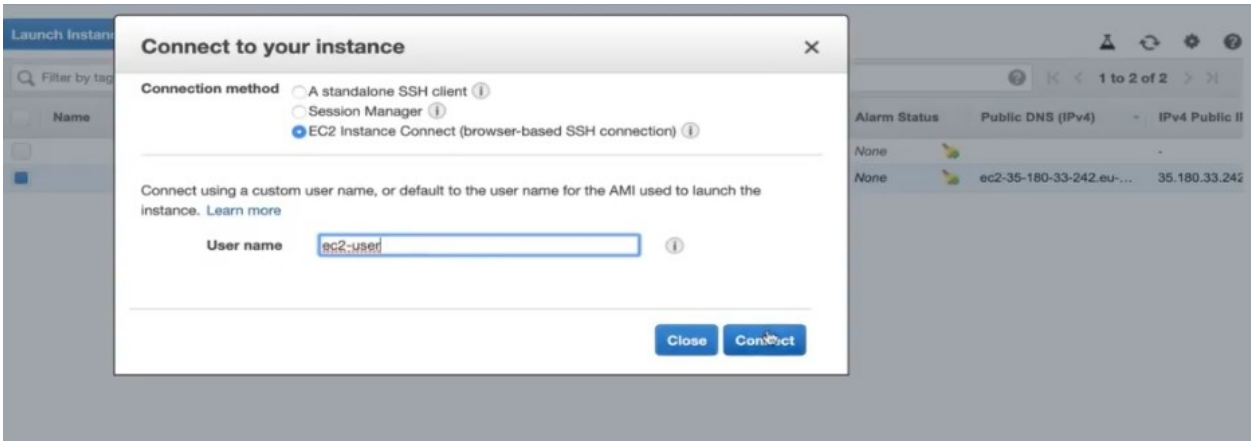
Then, an instance type needs to be chosen. Note that because of Rancher, it has to have at least 2GB Ram:



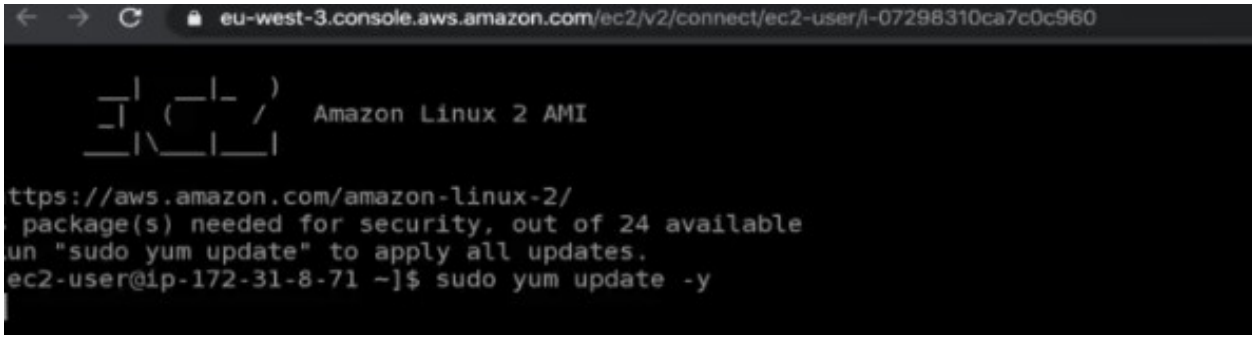
Now, some network rules needs to be configured in order to expose some ports so the airflow server can be accessed:



Now that it has been created, it can be accessed through an EC2 instance connection (just because its easier to configure intially):

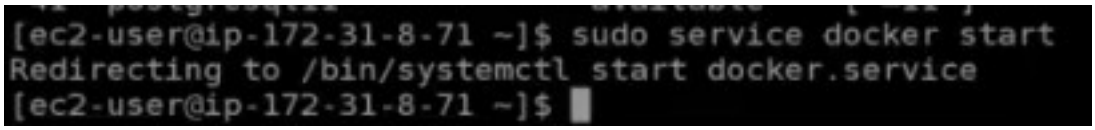


When connected, the first thing to be done is to make sure every package is up to date:

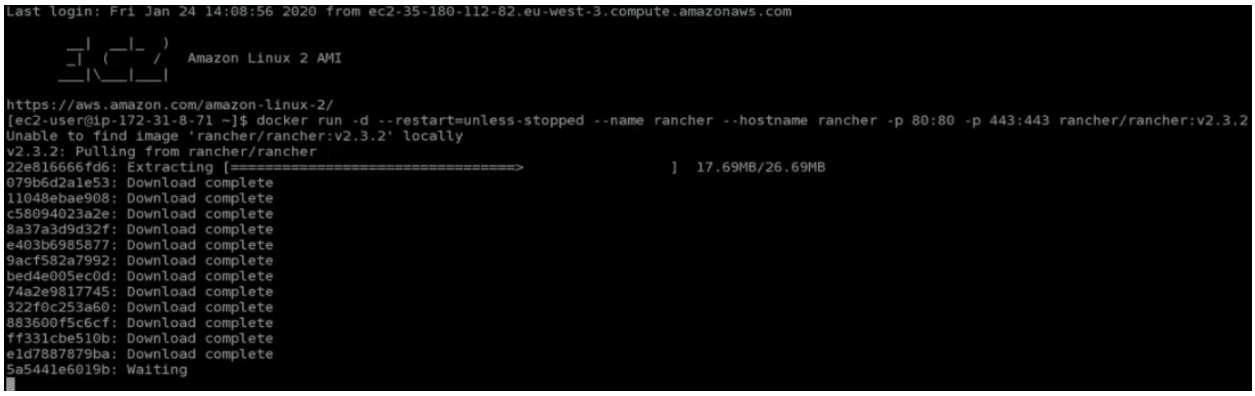


Then, install docker:

Starting the docker service:

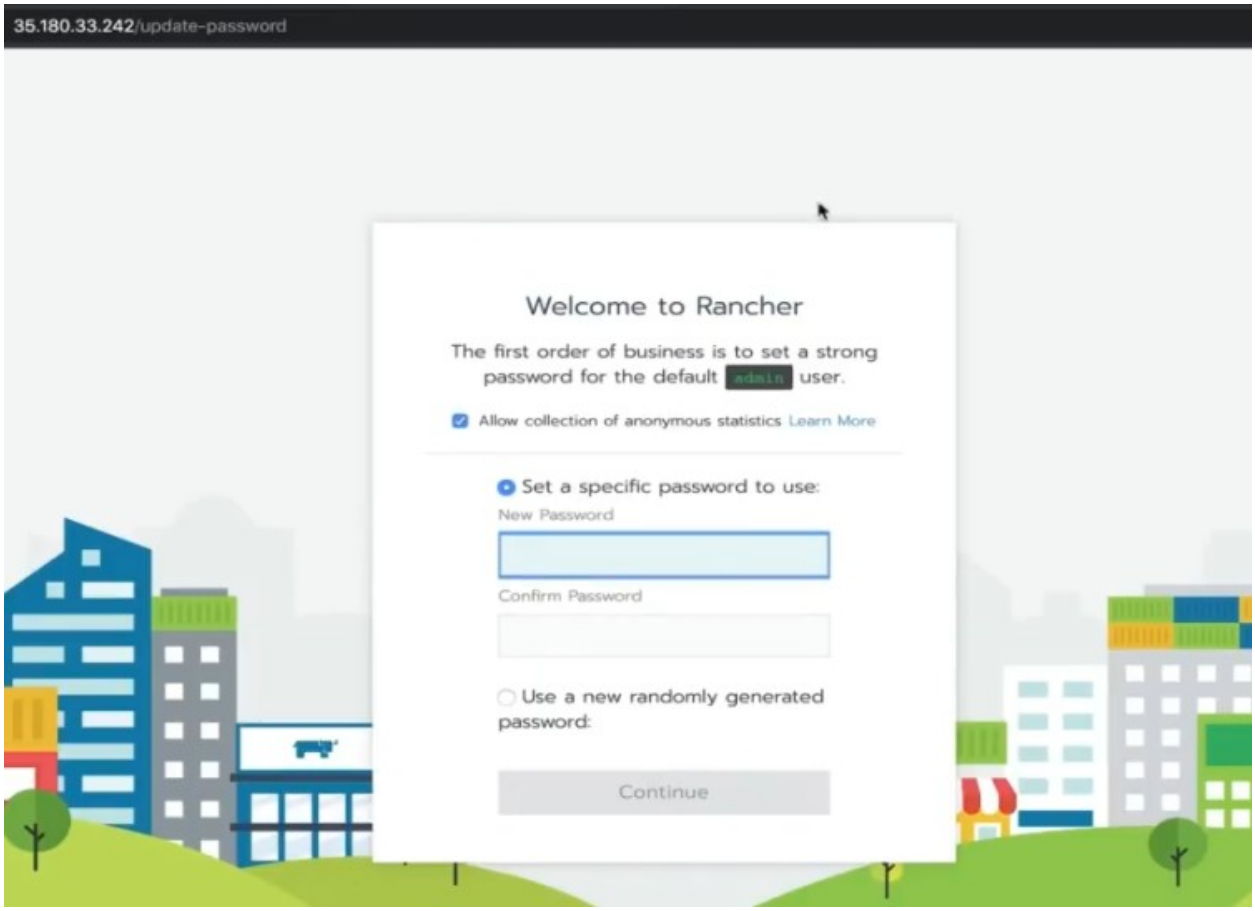


After the docker service is started, just run a new docker using the rancher image (since it doesnt come installed within the machine, it has to be installed):



After it, just access the exposed IP:

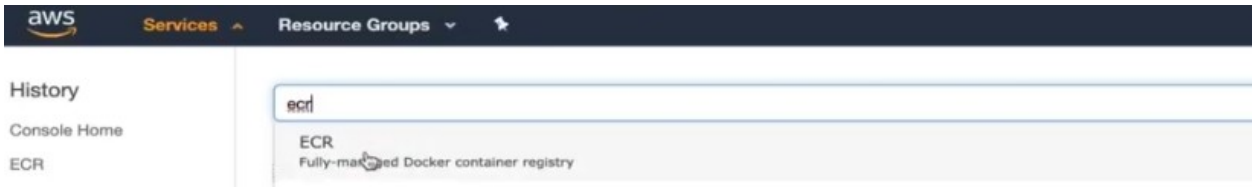
Now, Rancher should be sucessfully running in the AWS machine:

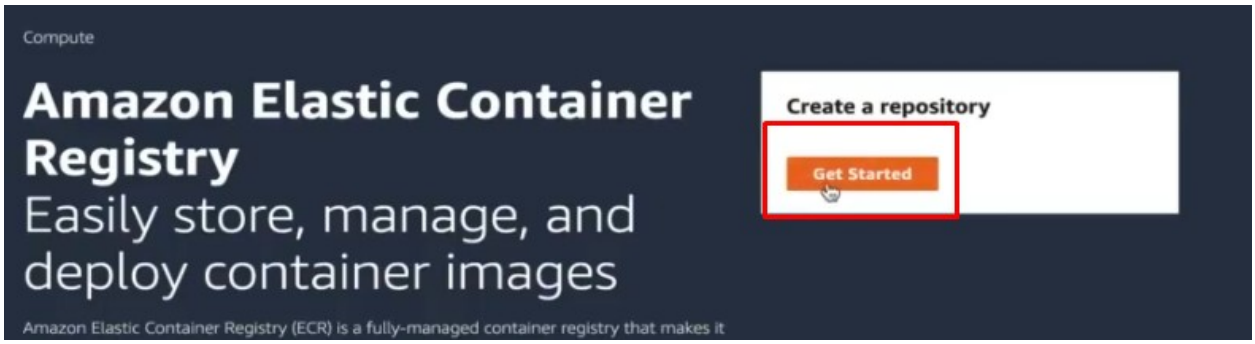


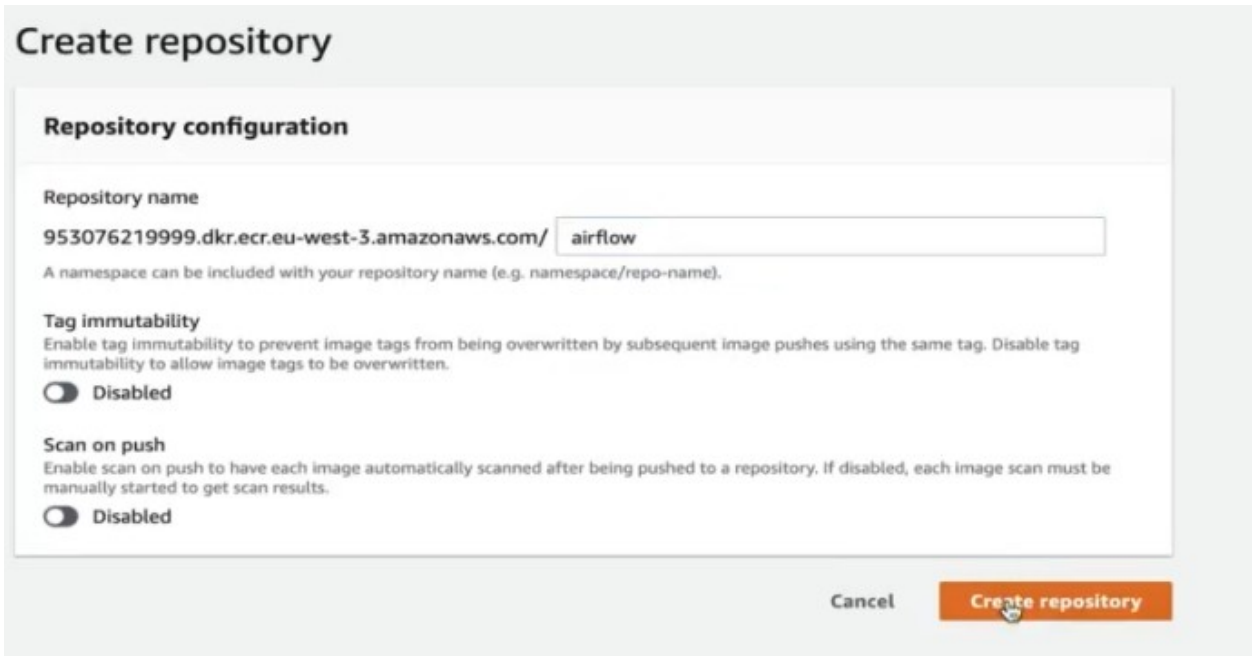
Creating an ECR repository

An ECR repository may be needed in order to store, manage and deploy docker containers.

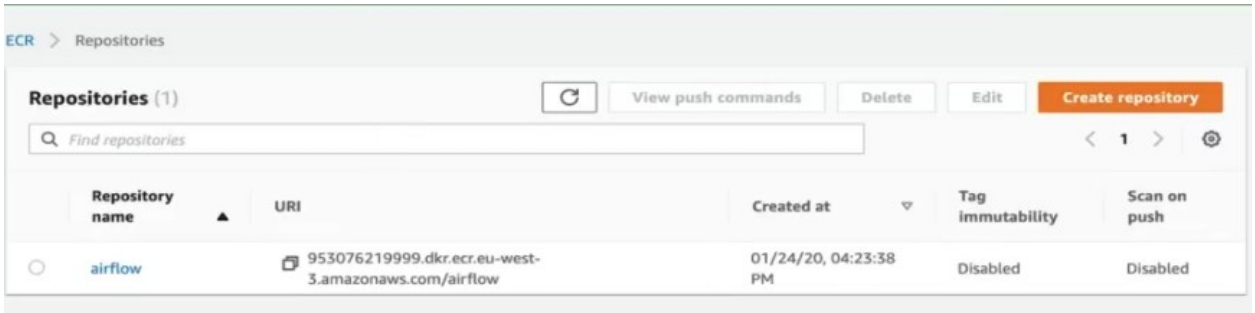
From the AWS console, search for ECR and then click on get started:



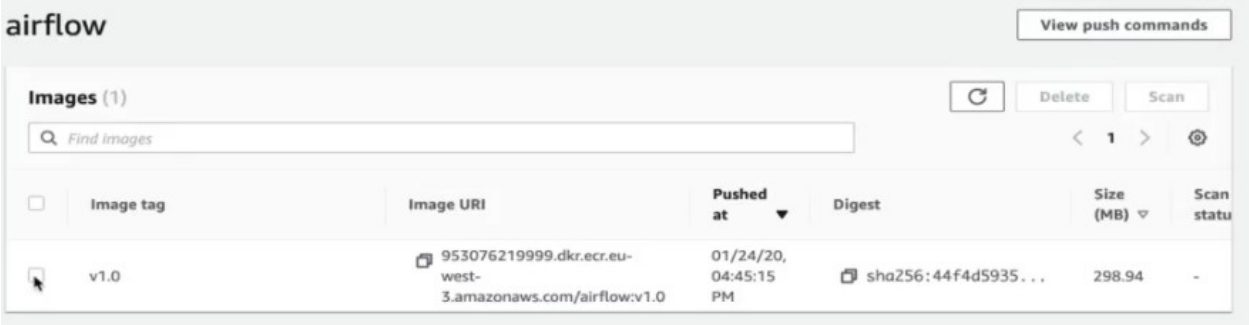




Now, the repository should be created:

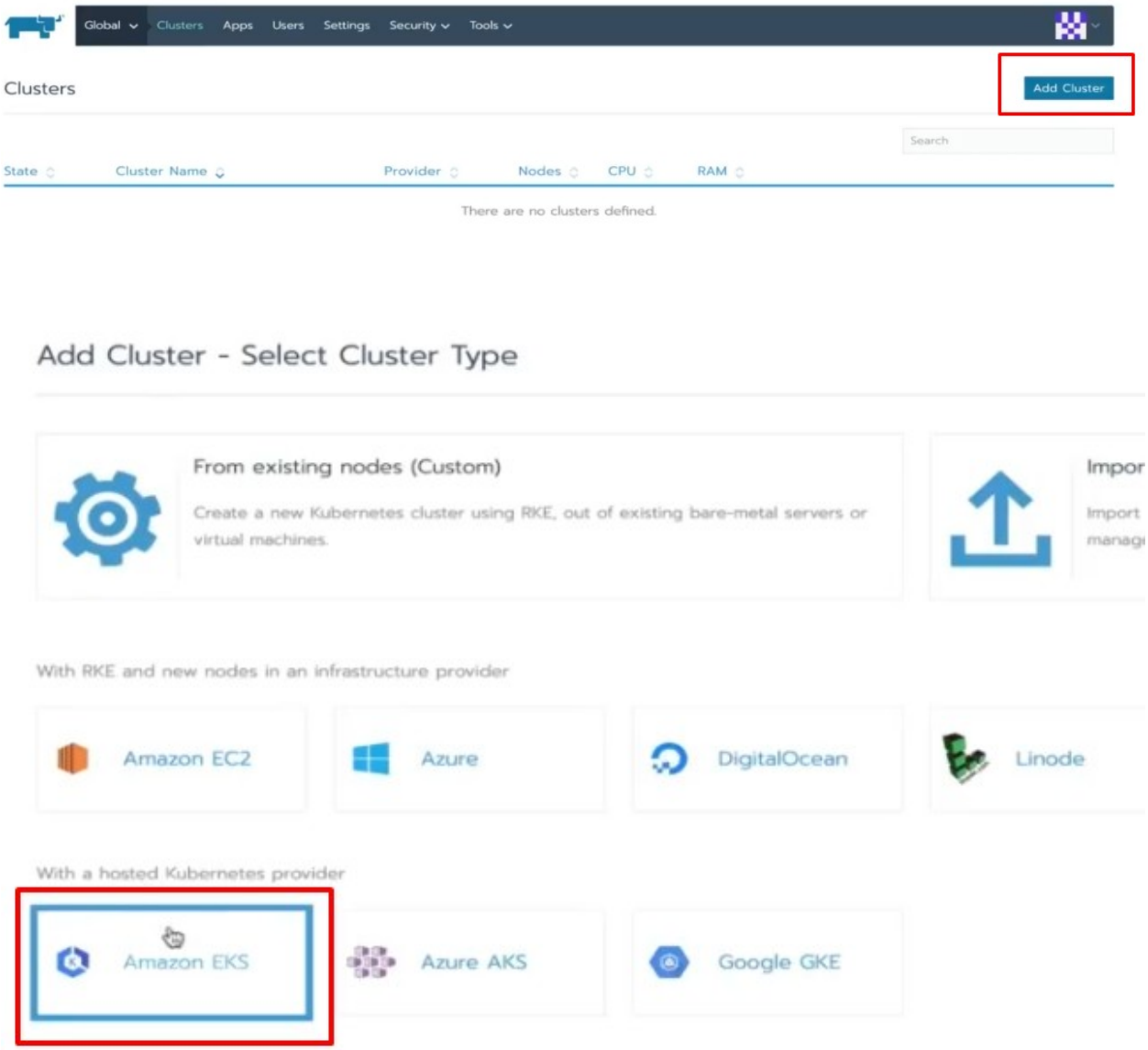


Now, docker images can be pushed into this repository, making it easier to use different custom images inside the AWS environment.

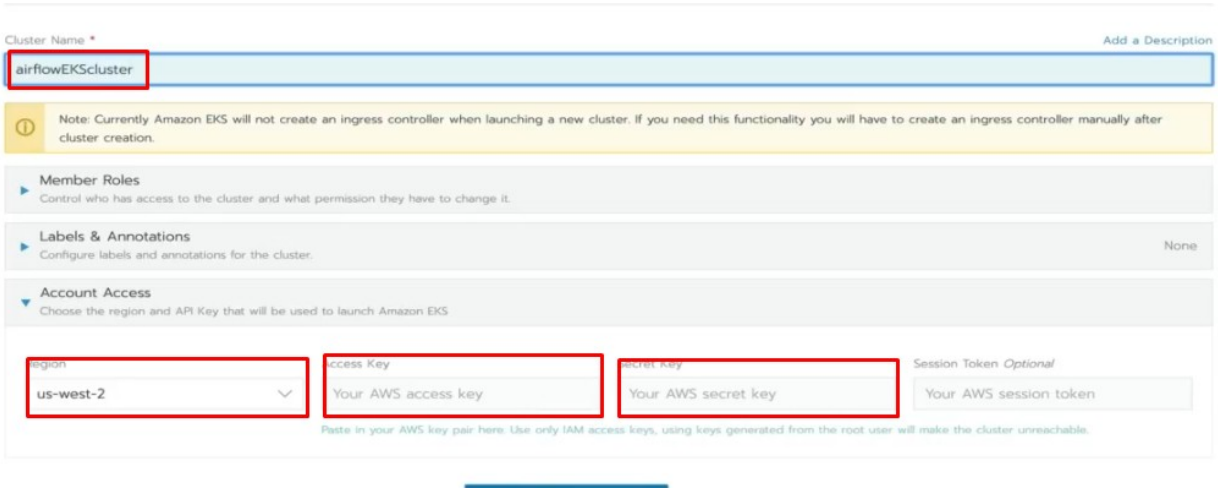


Creating an EKS Cluster with Rancher

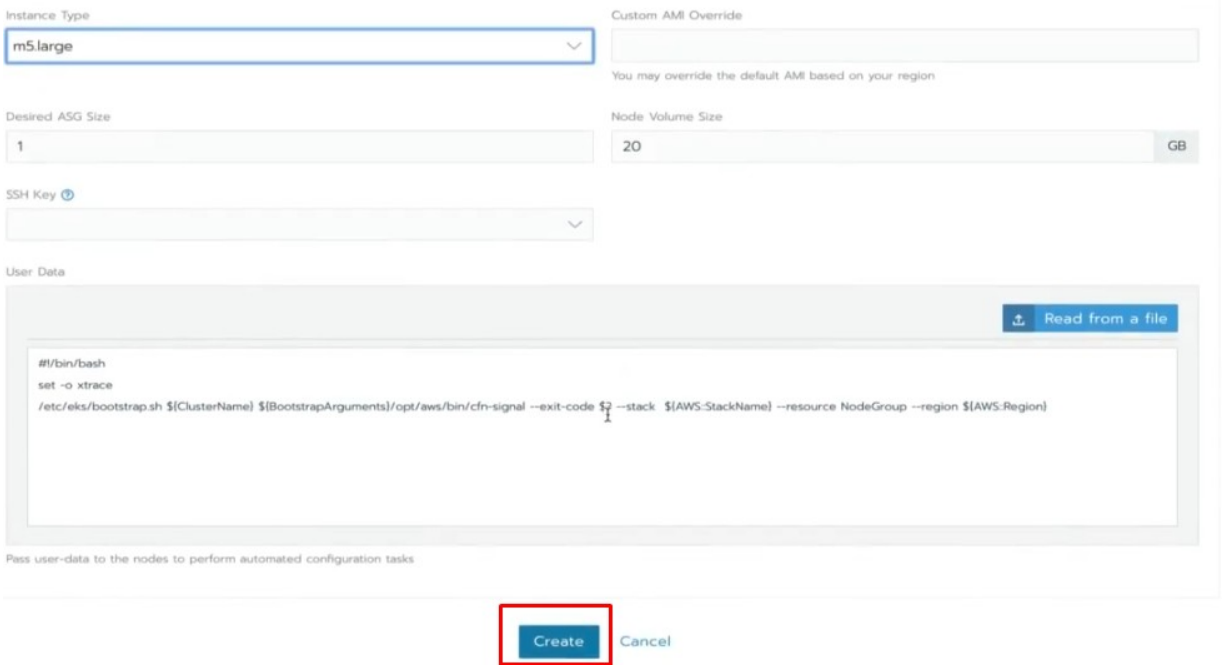
Instead of importing a cluster, a new one will be created:



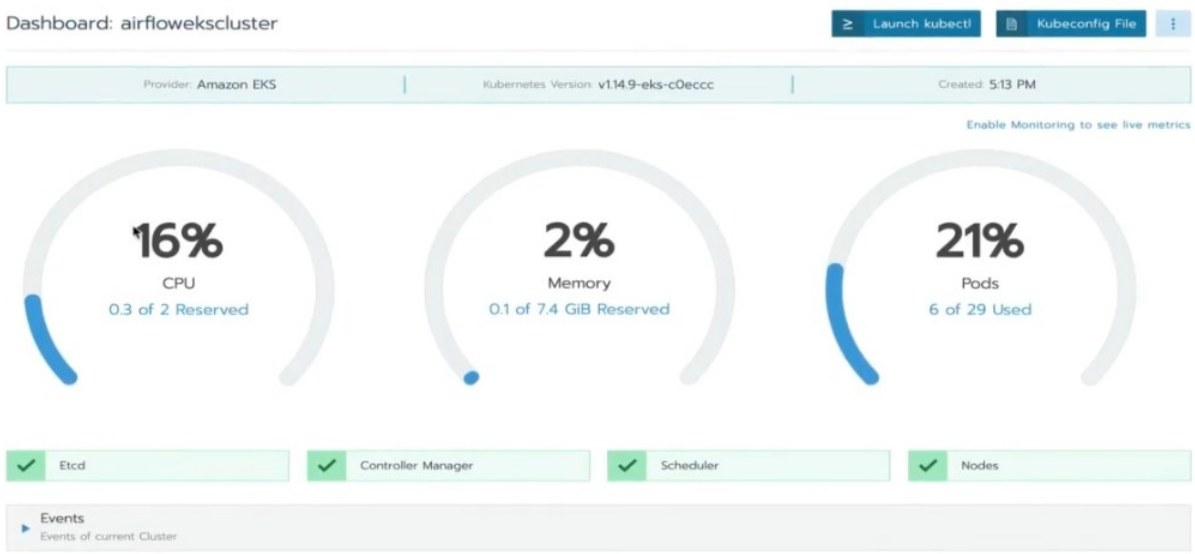
Now, some AWS security credentials need to me filled up in order to continue configuring the cluster: region, access key and secret key, which were provided by AWS when its instance was initiated:



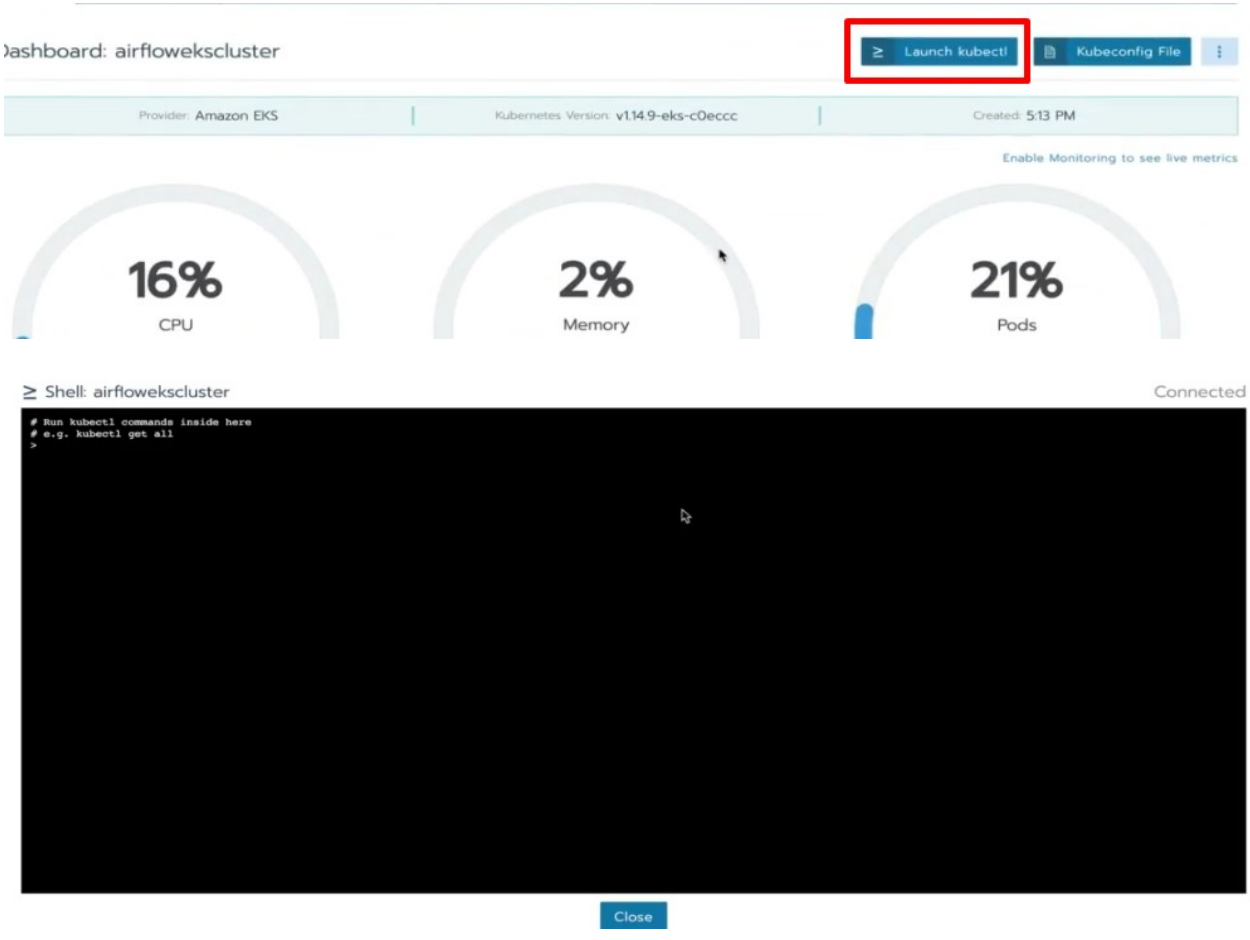
The amount of nodes and other kinds of similiar configuration can be configured before creating the cluster, but for the sake of simplicity, will be used the default settings. Click "Create":



After the cluster is created, a dashboard can be accessed to visualize how its using the machine resources:



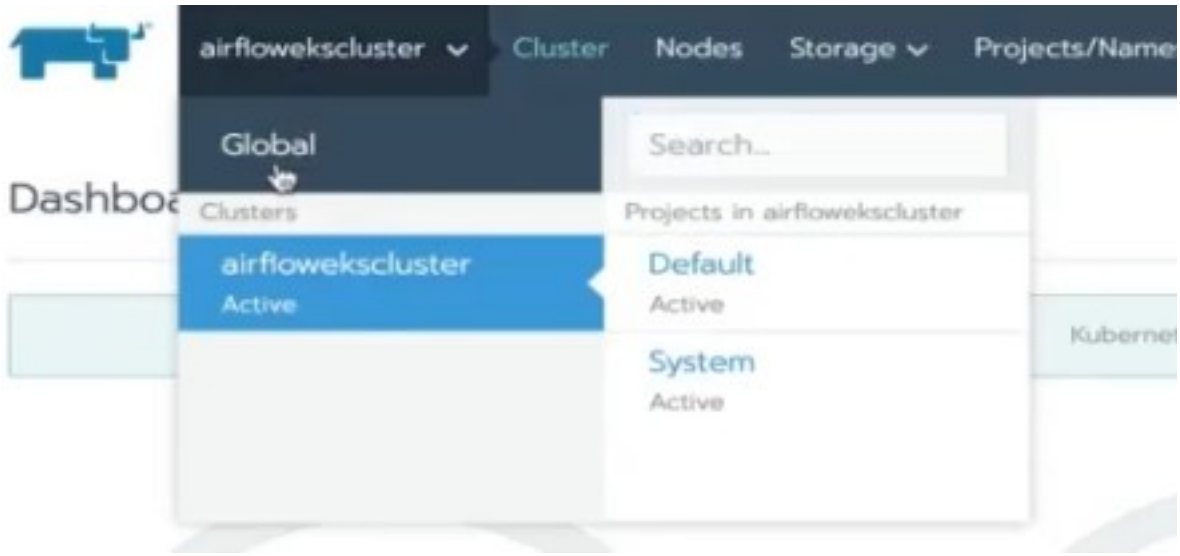
Launching a kubectl session



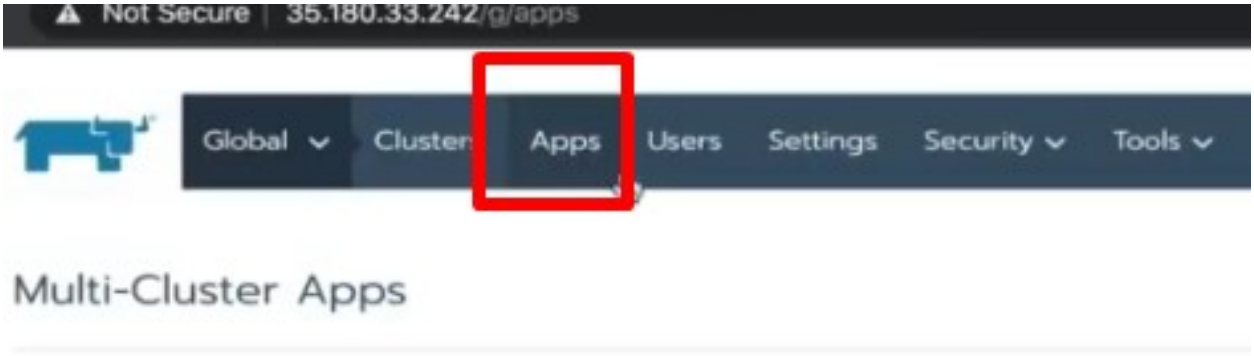
With this, its possible to execute any kubectl command into the cluster. For example: `kubectl version` (for version), `kubectl get nodes` (list all the nodes being used by the cluster), etc.

Deploy Nginx Ingress with Catalogs (Helm)

Firstly, instead of selecting a single cluster, set it to global

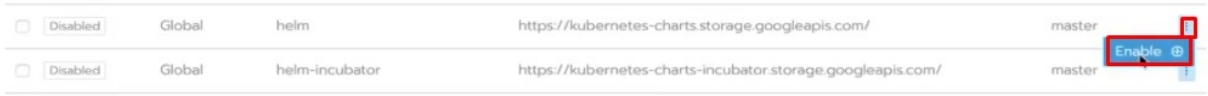


Then, click on "Apps":

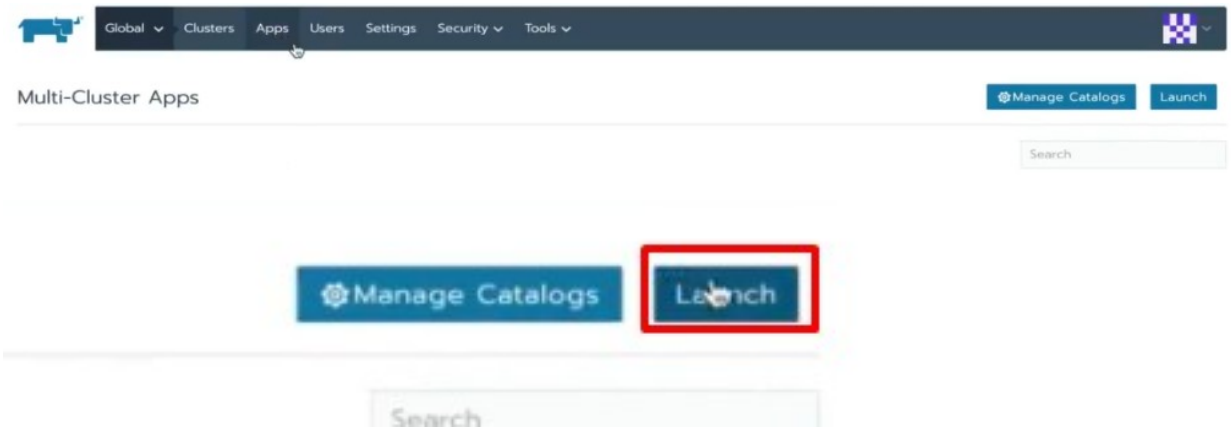


Then, "Manage Catalogs":

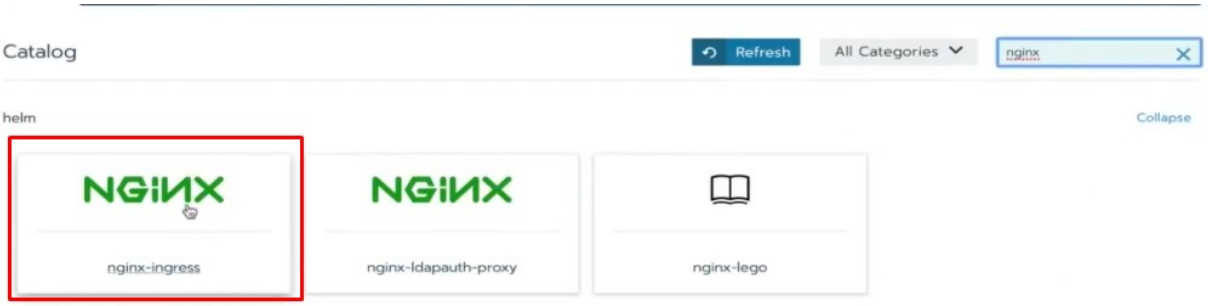
Now, enable the "helm" catalog:



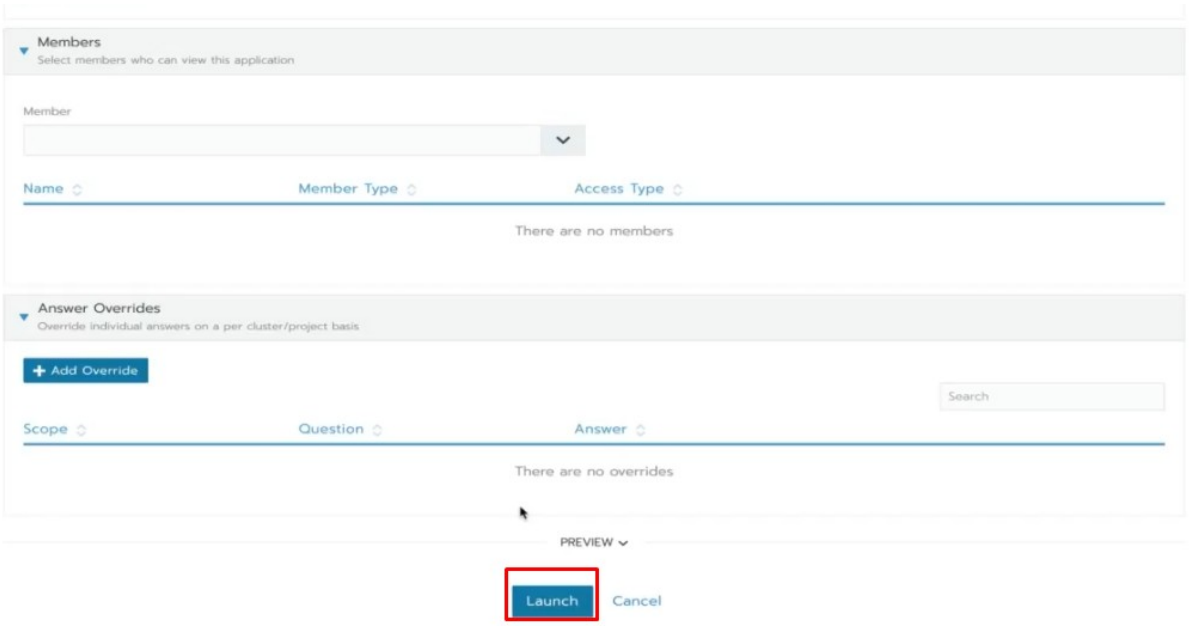
After it, it becomes active and starts to synchronize all the hand charts with Rancher. Once the sync is over, click on Apps again and then on Launch. With this, its possible to select many applications to be easily installed into rancher by using catalogs:



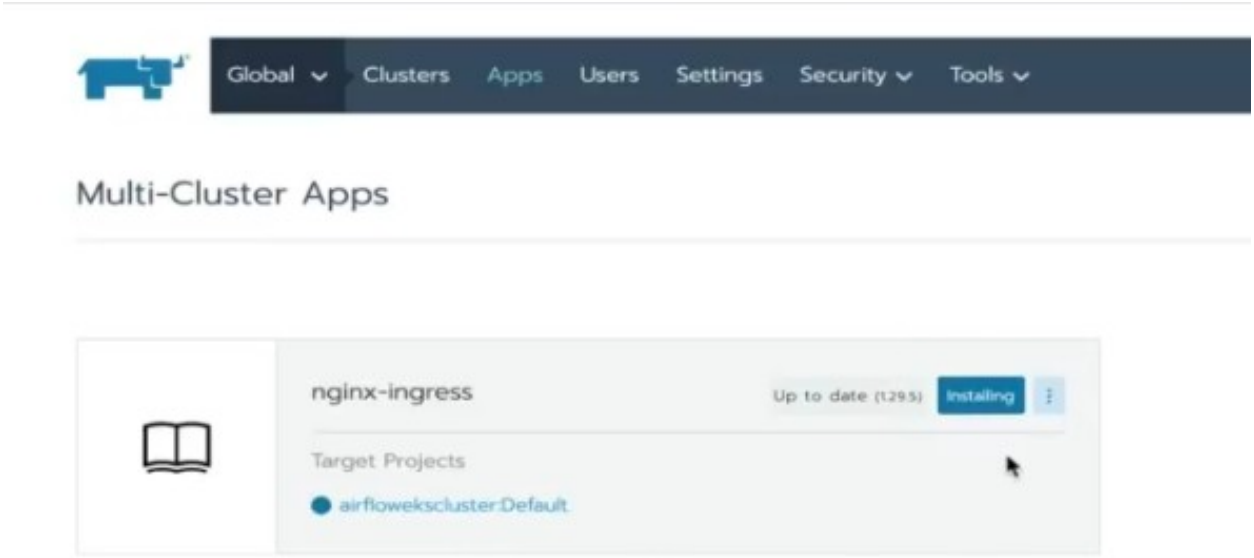
To install Nginx, just search for it then click on it:



After it, a page full of nginx configurations will pop up. After everything is configured nicely (will let the default settings, once again, for the sake of simplicity) and finally, click on "Launch":



Now, it should already appear as one of the Apps:

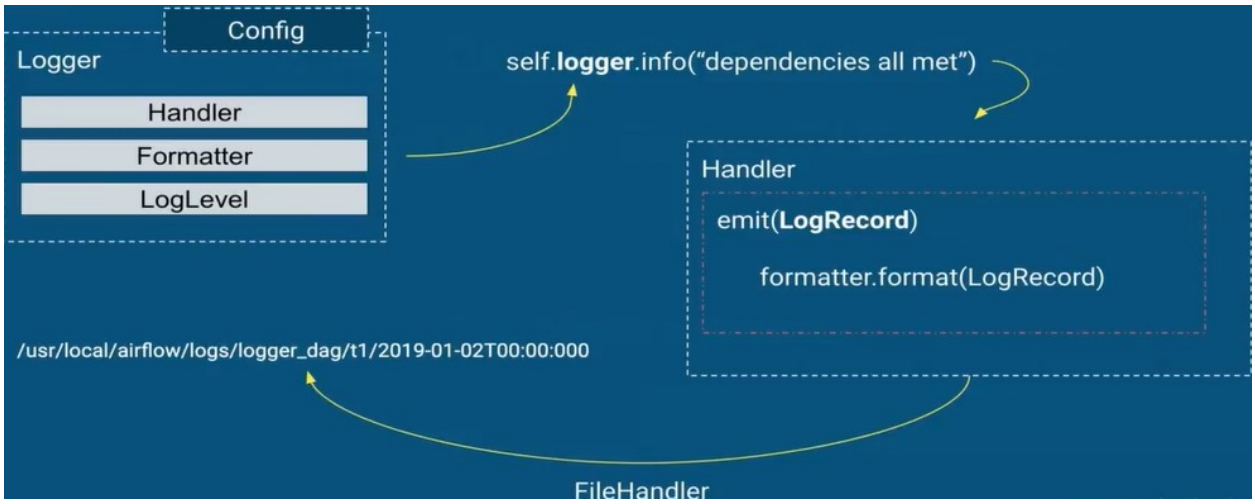


When clicked, will exhibit all the details related to its configuration and execution.

Monitoring Apache Airflow

How the logging system works in Airflow

- Based on the logging module
- Written into files
- Log levels (INFO, ERROR, DEBUG, WARNING, CRITICAL)
- Formatters
- Handlers for outputs:
  - FileHandler
  - StreamHandler
  - NullHandler



Setting up custom logging

In the airflow.cfg file, there are some important parameters to be set:

- `base_log_folder` is the parameter for setting the location where the logs will be stored.

```
# The folder where airflow should store its log files
# This path must be absolute
base_log_folder = /usr/local/airflow/logs
```

- `logging_level` is the parameter for setting which kind of information must be kept in the logs. By default, the value is set to `INFO`, meaning that any kind of information will be stored into the log file. If this is set to `WARNING`, only WARNING log levels will be stored into the log file.

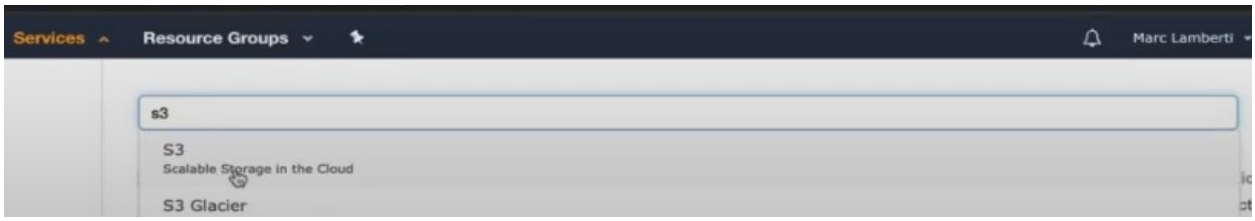
```
# Logging level
logging_level = INFO
fab_logging_level = WARN
```

- `log_format` defines the log format.

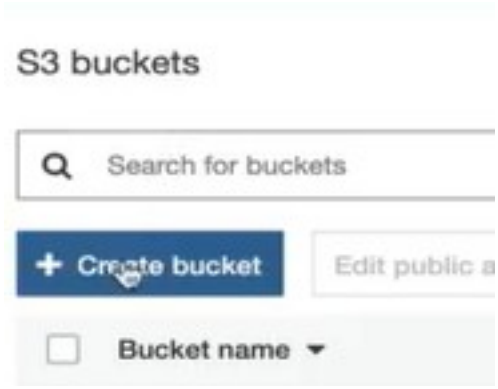
```
log_format = [%%(asctime)s] {%%(filename)s:%%(lineno)d} %%(levelname)s - %%(message)s
simple_log_format = %%(asctime)s %%(levelname)s - %%(message)s
```

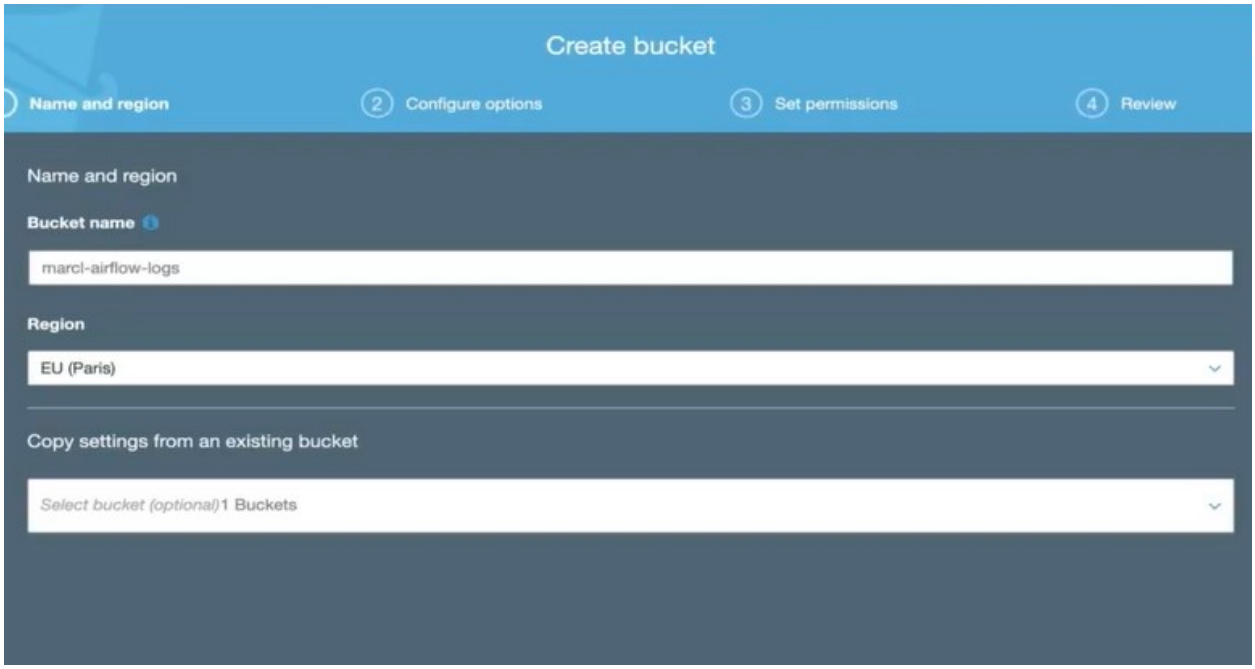
Storing logs in AWS S3

It's also possible to store airflow logs on cloud services, like AWS S3. Firstly, search for S3 in the AWS dashboard:

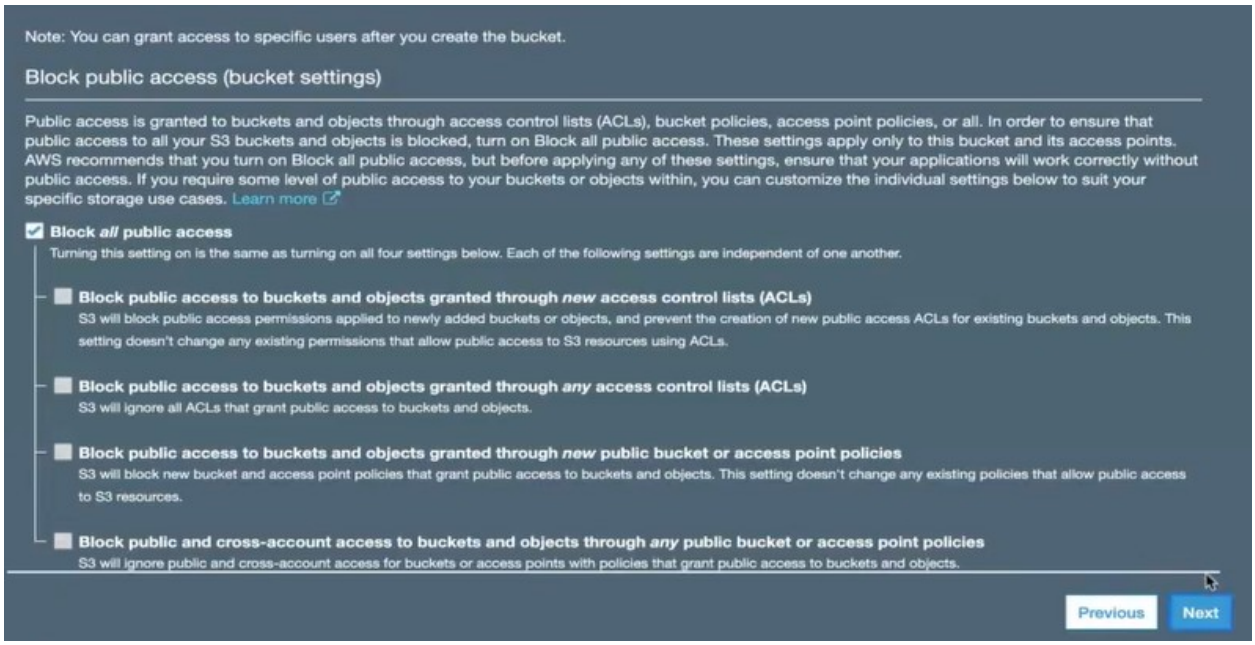


Then, click on "create bucket":

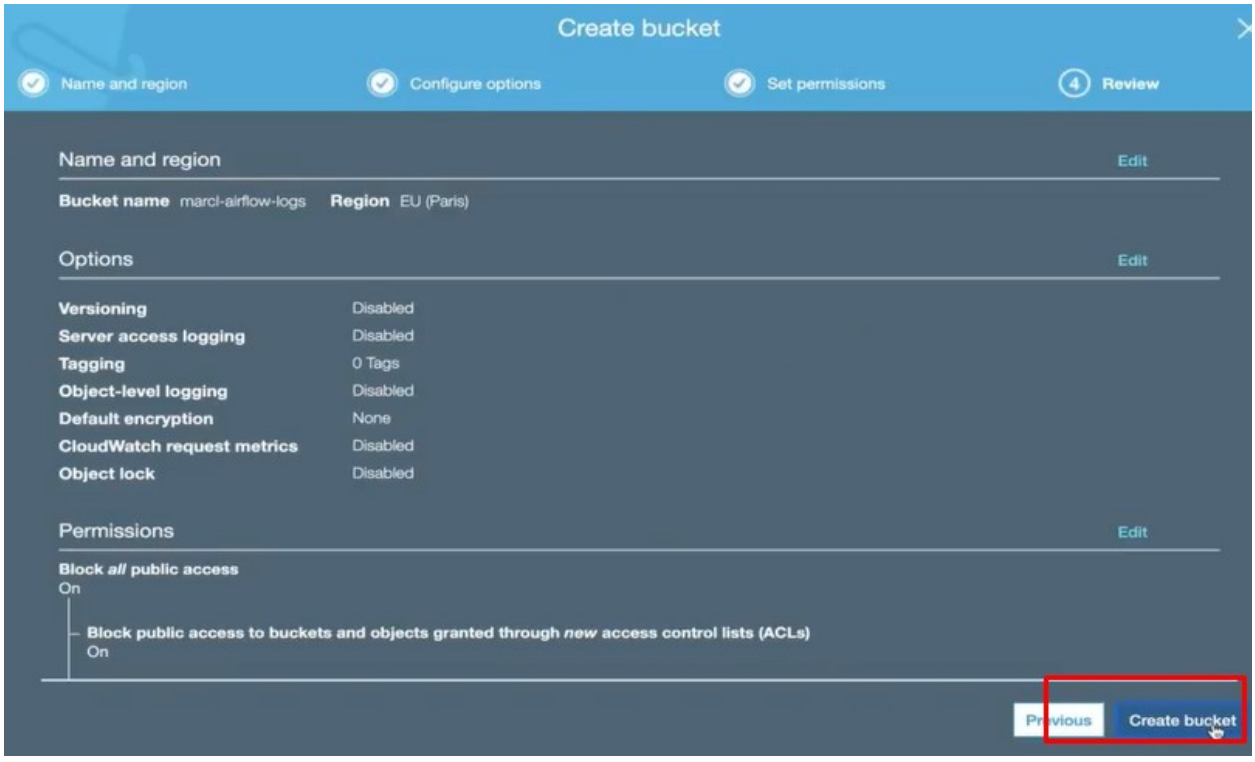




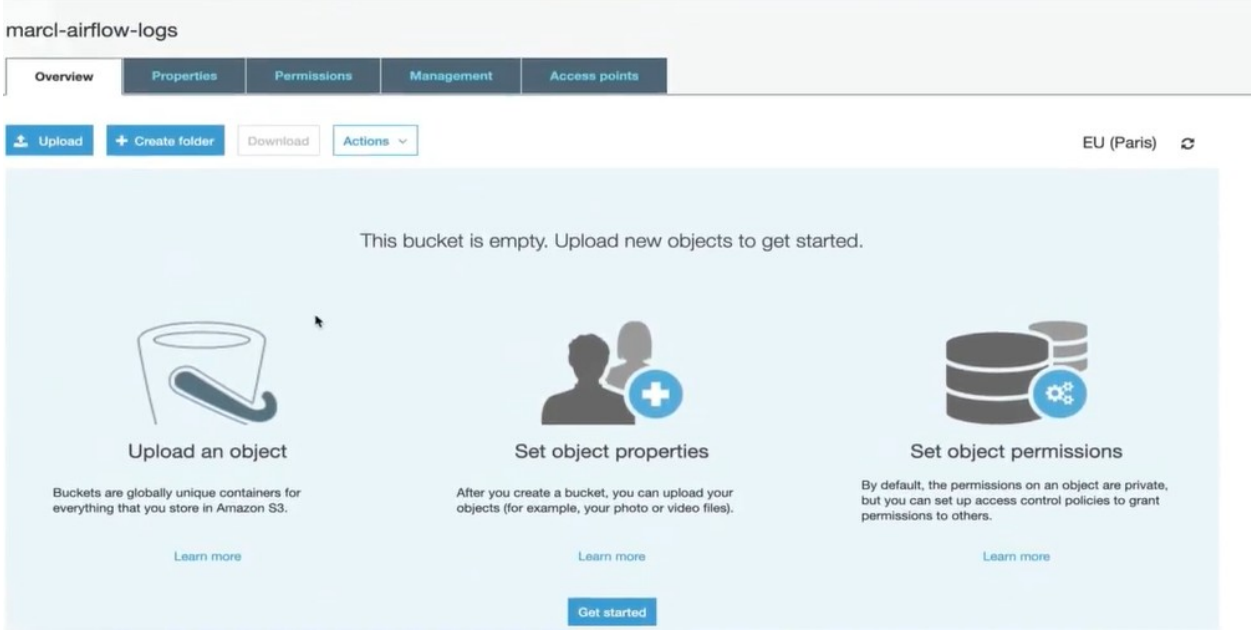
Make sure its set to public access:



Now, create the bucket:

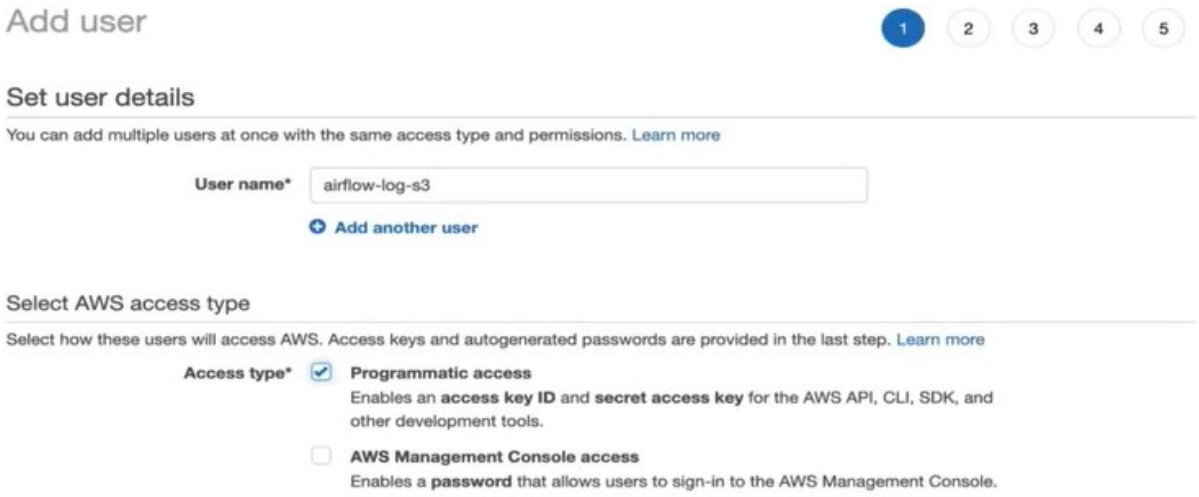


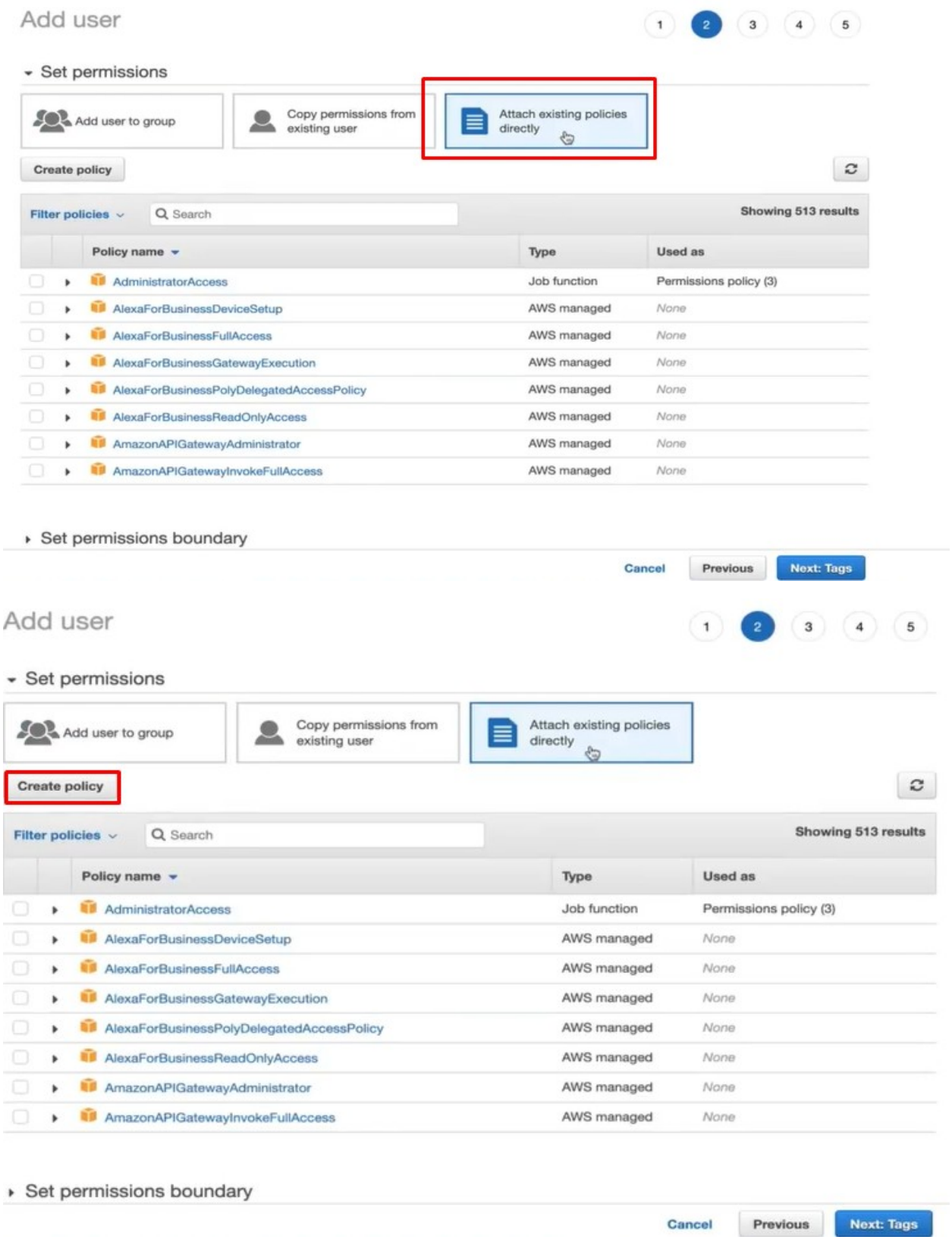
Now, there should be an empty bucket ready to use



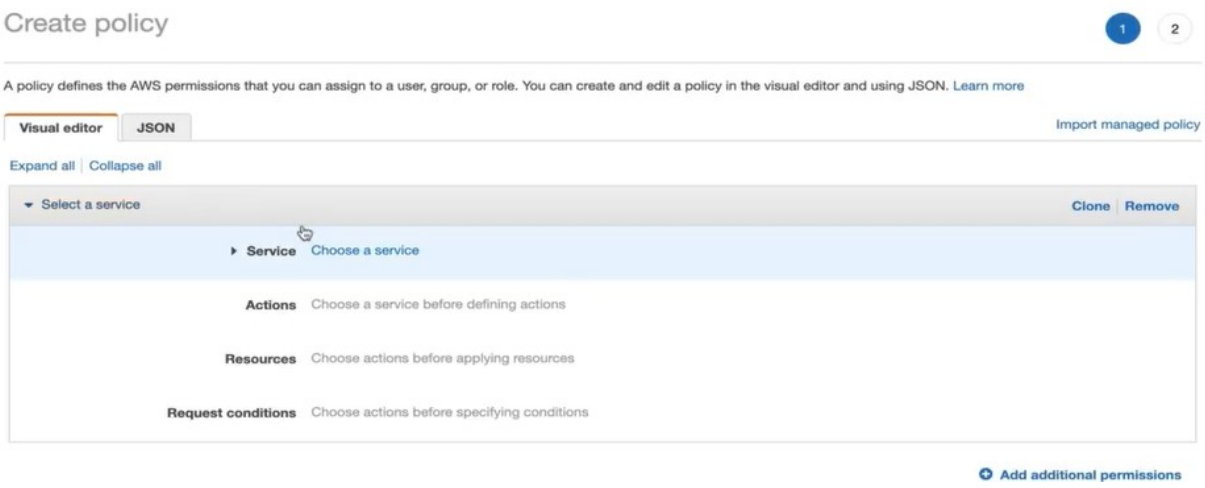
Setting up an user to access and modify data on this bucket

Here, a new user will be created, as well with its configuration for optimal access to the bucket.

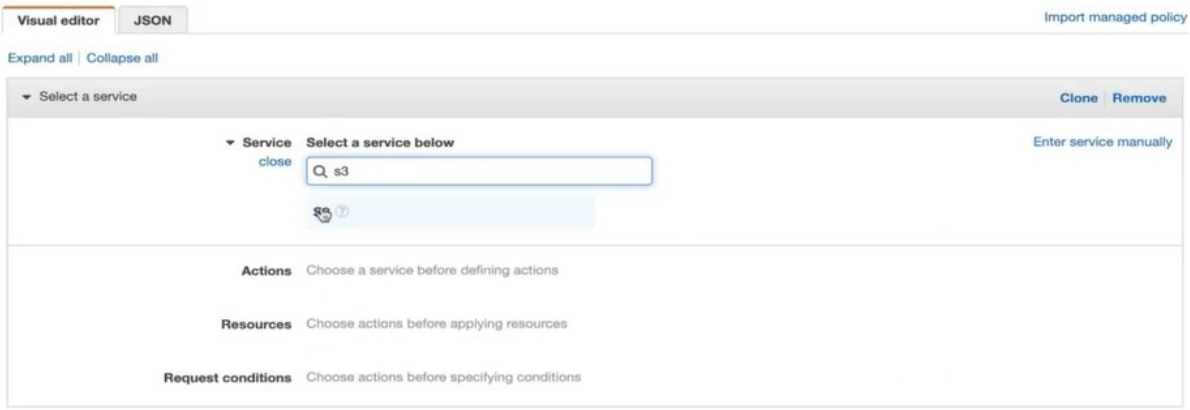




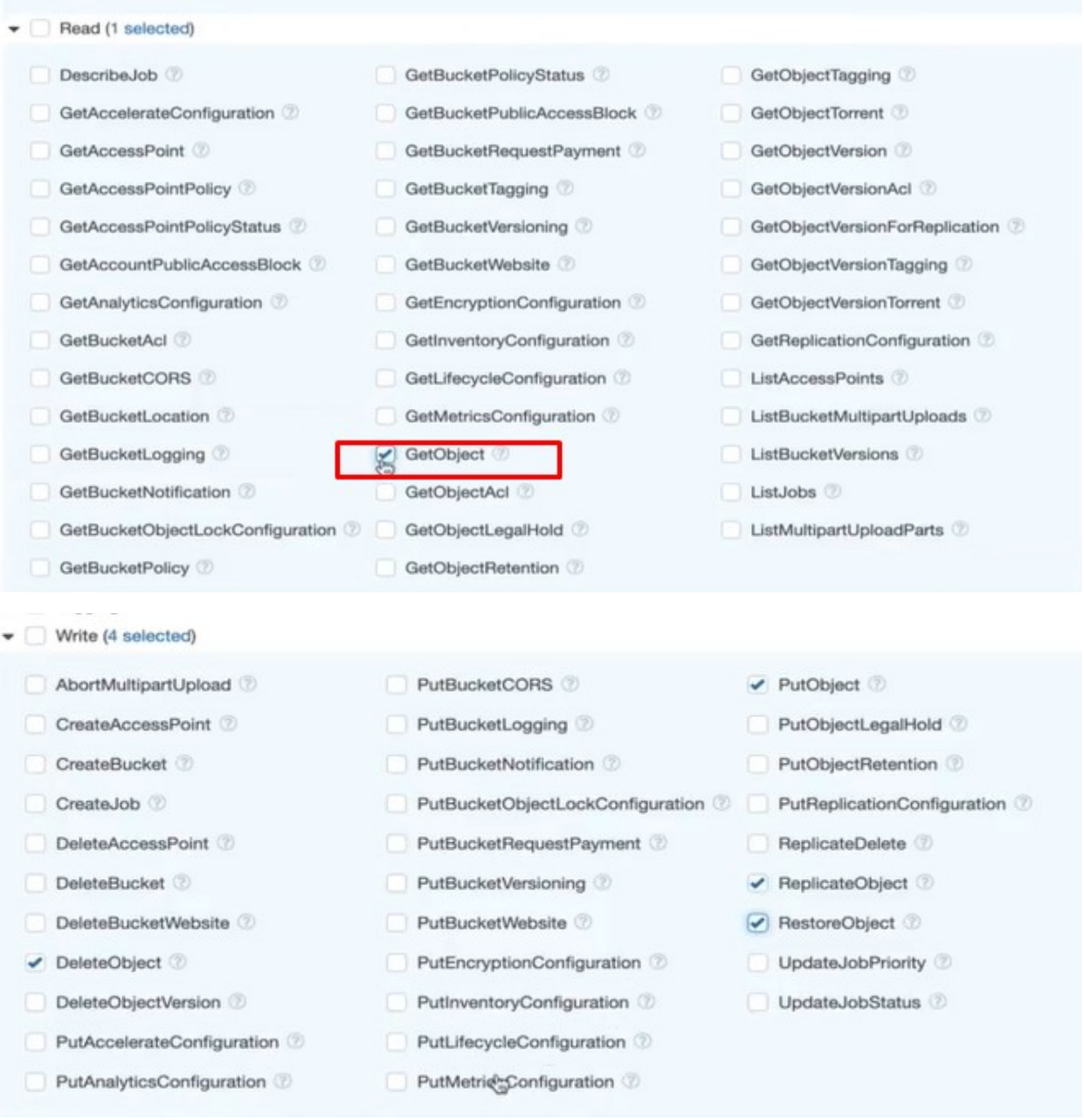
Now, a new policy will be created for this new user's usage:



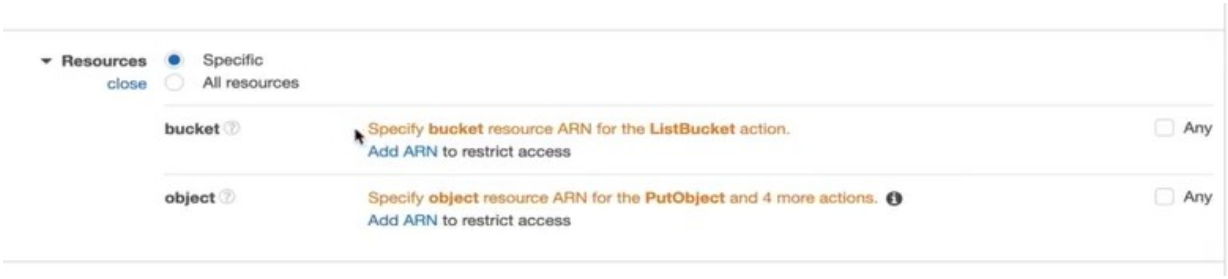
For "Service", chose S3:



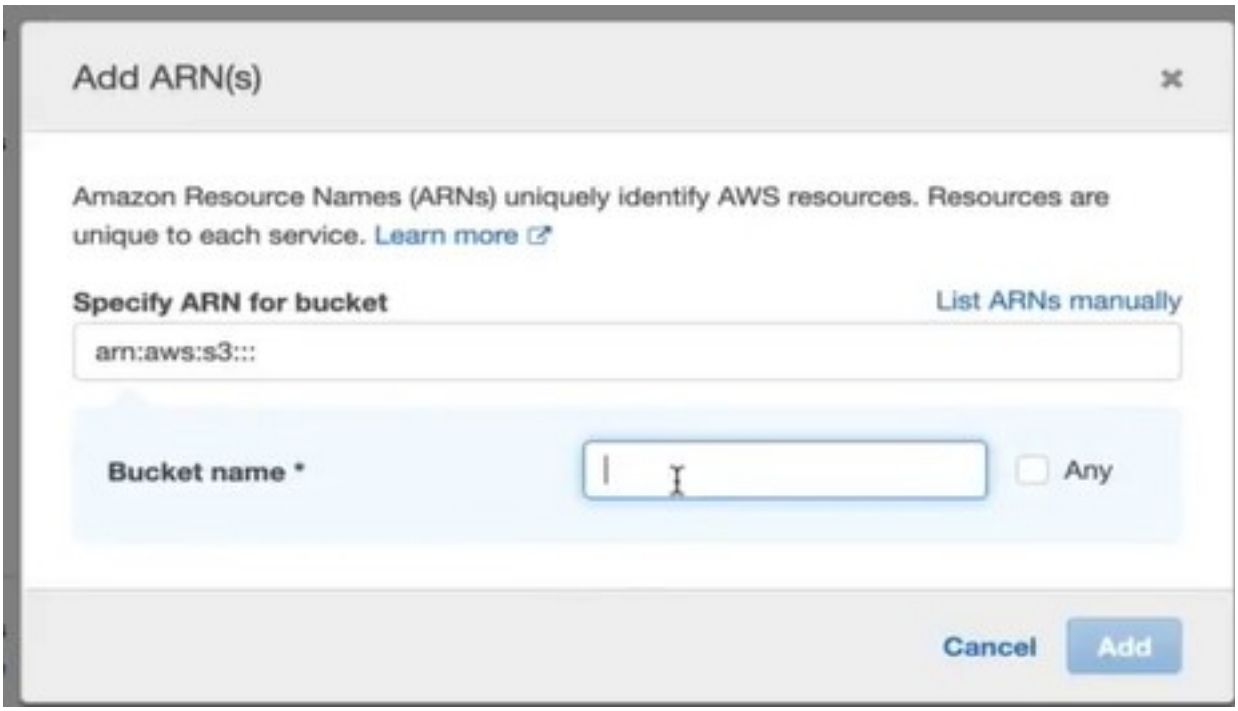
For "Actions", chose ListBucket, GetObject, PutObject, DeleteObject, ReplicateObject and RestoreObject.

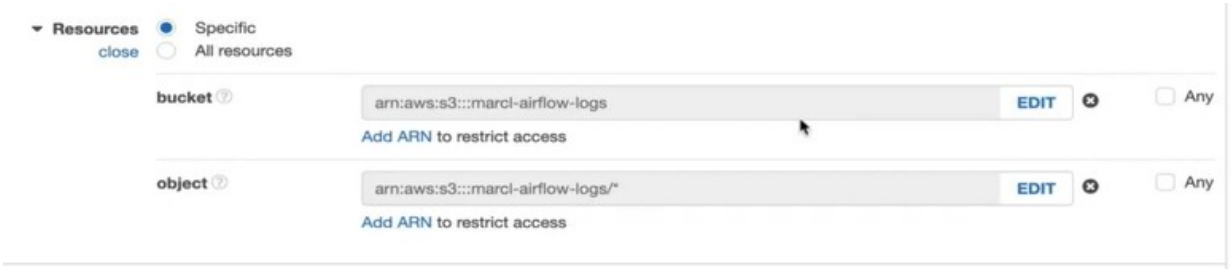


Now, the created bucket must be selected in the Resources section:

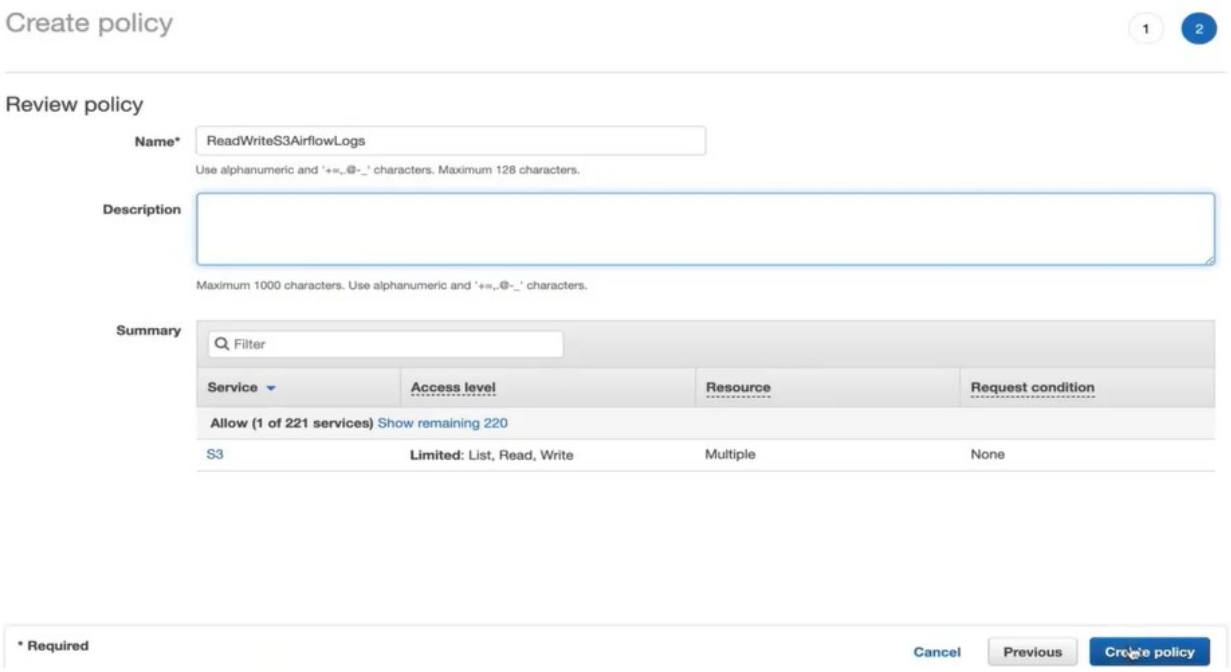


Fill in with the desired bucket name:

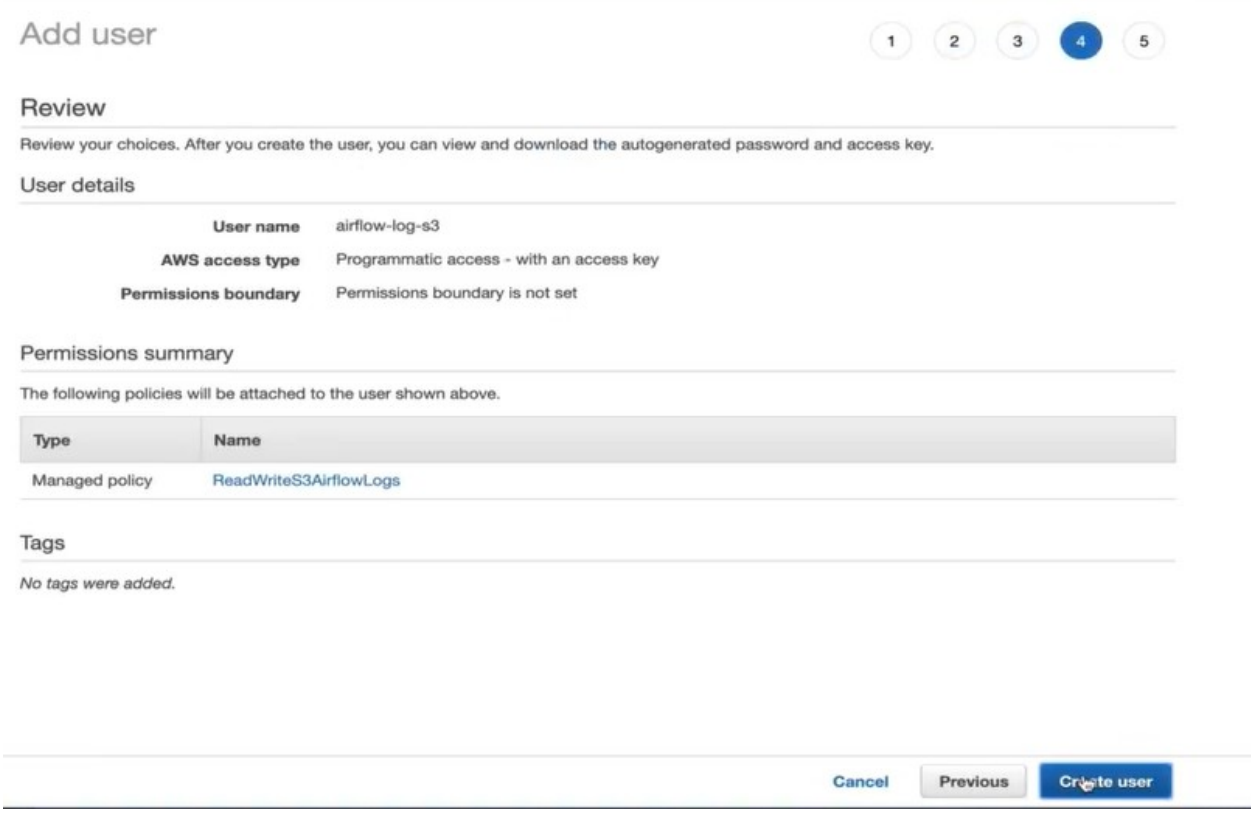




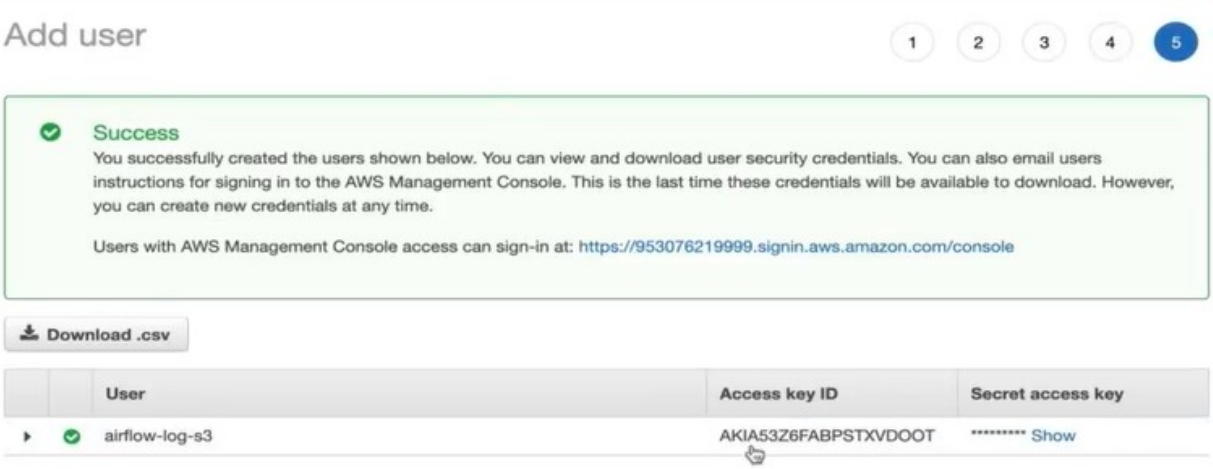
Now, just set up a name for this policy and finish its creation:



Now that the policy has been successfully created, it should be attached to the new user:

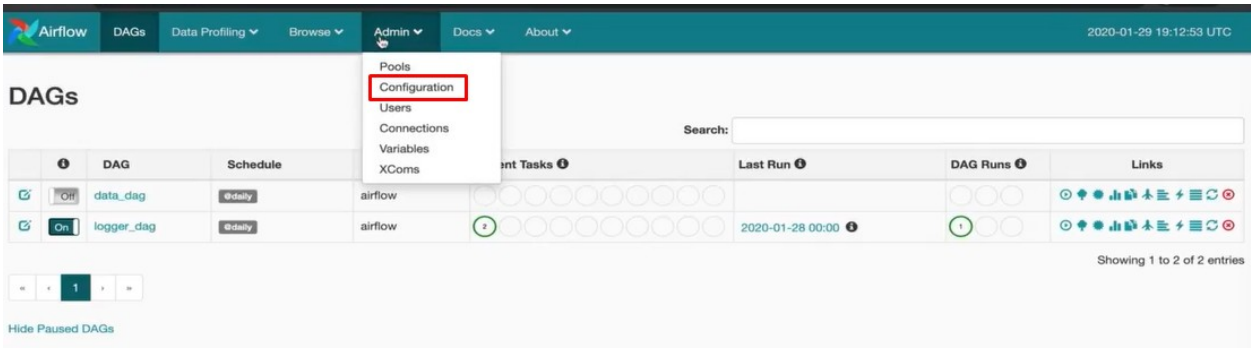


Now, this new user should now be able to add, modify and remove data from the bucket created.

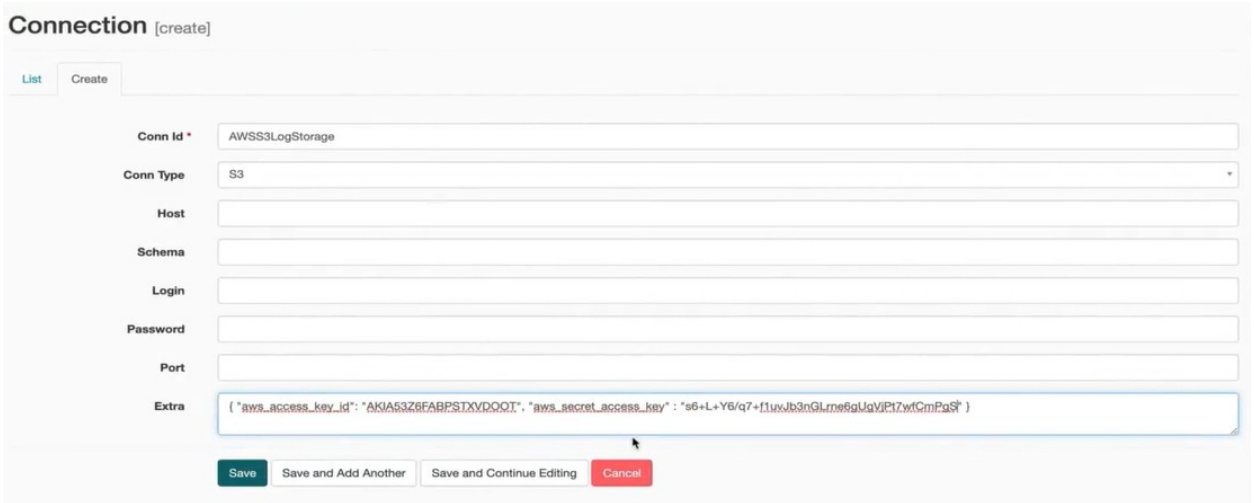


Completing the connection on airflow

Now that everything is set up, its only needed to complete the connection by creating a new connection on airflow:



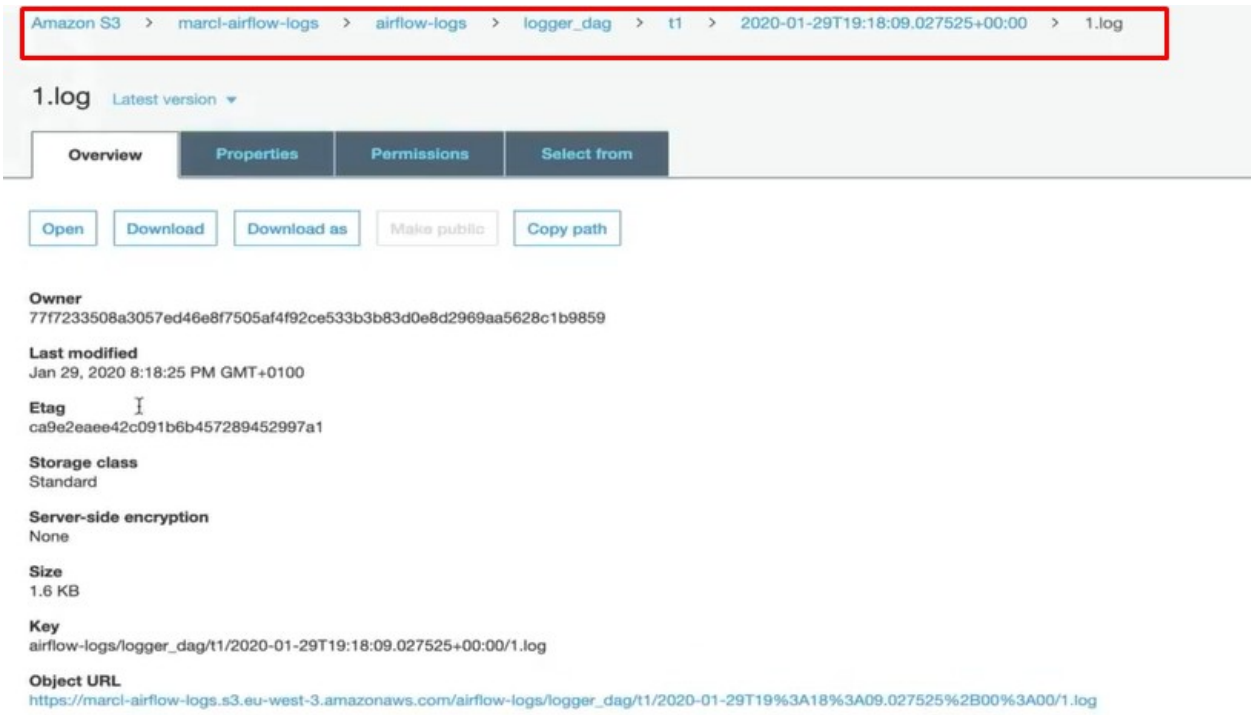
Now, set up the connection, specifying the AWS access key and secret acess key:



Now, in the airflow.cfg file, enable remote access, specify the name of the connection responsible for making remote access, as well as the directory in which the logs must be saved, which, in this case, will be the bucket created previously.

```
# Airflow can store logs remotely in AWS S3, Google Cloud Storage or Elastic Search.
# Users must supply an Airflow connection id that provides access to the storage
# location. If remote_logging is set to true, see UPDATING.md for additional
# configuration requirements.
remote_logging = True
remote_log_conn_id = AWSS3LogStorage
remote_base_log_folder = s3://marcl-airflow-logs/ariflow-logs
encrypt_s3_logs = False
```

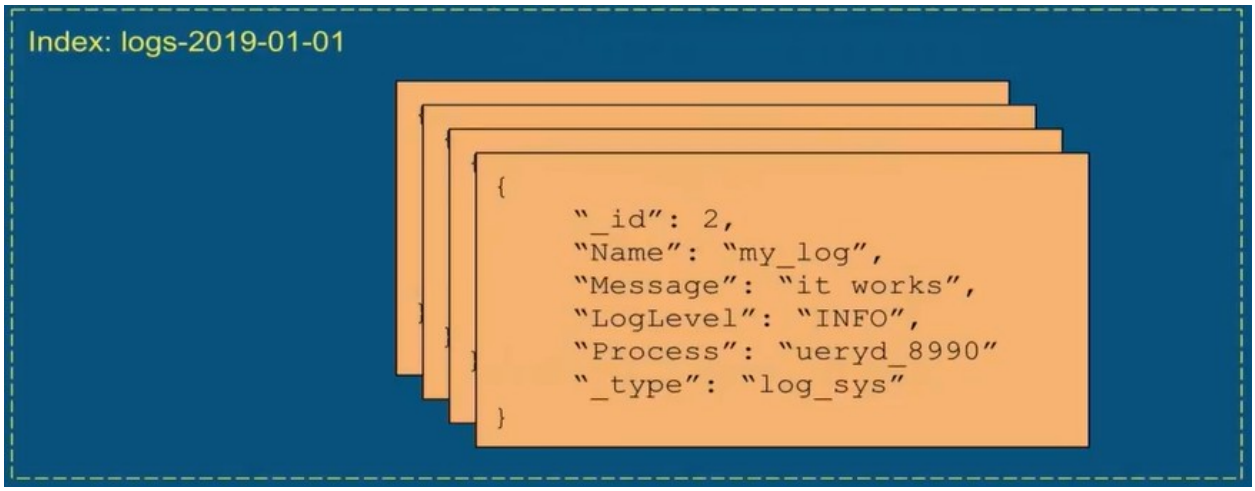
Now, every log generated by airflow will be automatically saved into the AWS S3 bucket:



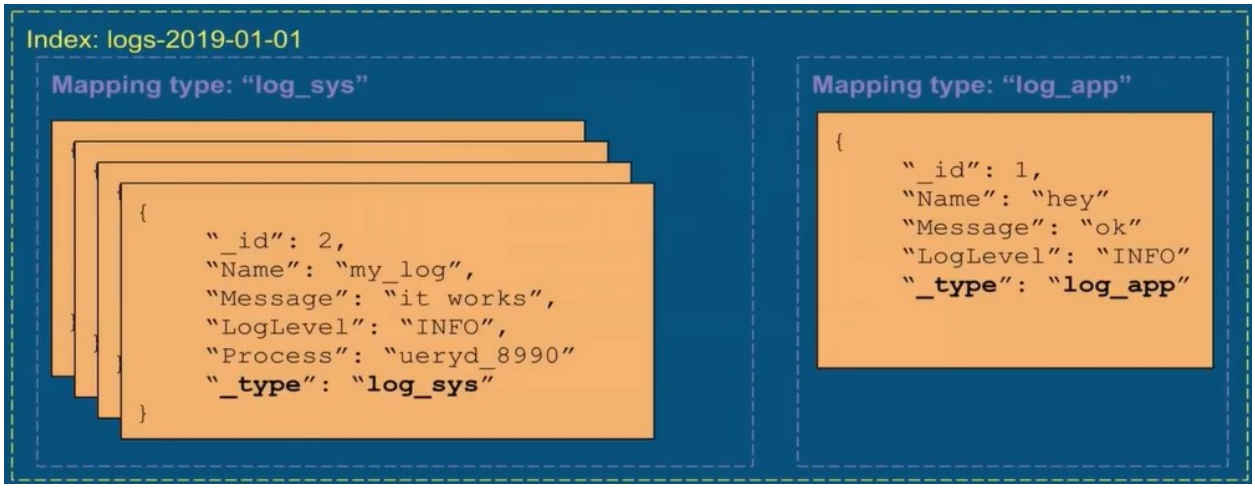
Elasticsearch

Elasticsearch is a document-oriented databased that works based on two principal concepts:

- Document: a document can be seen as row in a relational database and it corresponds to your data stored in json format in elasticsearch, so basically a document could be the following json data coming from a log file with different fields such as id, name, and log level, as well with their associated values. Since your log file contains many log events, each log event is a document along with an unique ID, then these documents are stored into an index, which is a collection of documents.



Then, an index can have one or more mapping types that are used to divide documents into logical groups inside the same index. A mapping defines how a document it indexed an how its fields those one are indexed and stored. You can think of a mapping as a table in a relational database, basically, each document has a defined type as you can see from the field `_type`. Finally, once your documents are mapped and stored into an index, you can start requesting them through the rest API given by Elasticsearch.



Introduction to metrics

Airflow updates different metrics such as the scheduler heartbeats to know if its still up, the time taken to scan and import all dag files at once, the number of running tasks on executor and so on. All of these metrics are sent to another tool called StatsD in order to export them.

So what is StatsD? StatsD is a simple daemon developed and released by hc in order to aggregate and summarize application metrics. You can think of StatsD as a push-based monitoring system where it receives logs from an application and push them to somewhere else such as elasticseaerch or influxdb, for example. Whenever airflow wants to send a metric, this metrics is sent over udp datagrams to the statsd daemon.

But why UDP? Well, since this protocol doesnt wait for acknowledgement like with TCP, its extremely fast and there is no risk of blocking the application waiting for StatsD to receive the metric.

- Airflow sends metrics do StatsD
- Daemon to aggregate and summarize application metrics
- Extremely fast (UDP) and tiny resource footprint
- Forward metrics to other applications

All metrics are based on three types:

- Counters (number of overall tasks failed,)
- Gauges (number of queued tasks on the executor)
- Timers (miliseconds taken to finish a task)

TIG Stack

- Telegraph
  - Agent for collecting, processing, aggregating metrics
- InfluxDB
  - Time series database
- Grafana
  - Data visualization and monitoring application

To connect Airflow to the TIG Stack, you can configure Airflow to export its metrics using the statsd or prometheus monitoring plugin. Telegraf acts as the data collector, receiving metrics from Airflow (via StatsD, Prometheus, or log parsing) and forwarding them to InfluxDB, a time-series database designed for storing and querying metrics. Grafana is then linked to InfluxDB as a data source, enabling visualization and monitoring of Airflow's performance and system metrics through customizable dashboards. This integration ensures seamless data flow from Airflow to Grafana via Telegraf and InfluxDB.

With this, it's possible to visualize how the data is behaving inside the airflow environment, making possible to better understand how the system is performing at different tasks.